EventHive

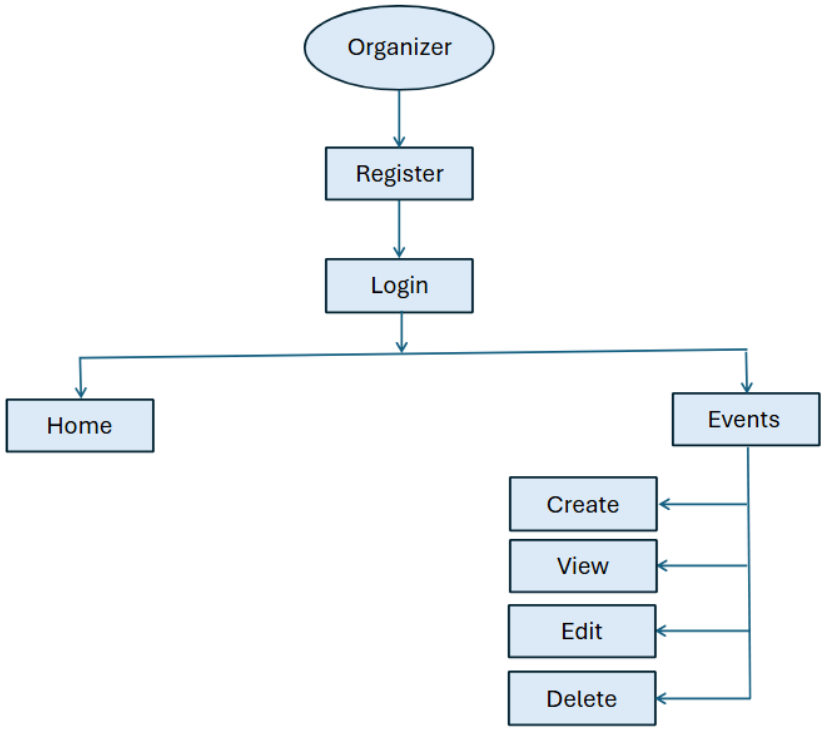
Introduction:

EventHive is a comprehensive event management platform designed to connect event organizers with attendees. The platform provides a seamless experience for discovering, creating, and managing events of all types - from conferences and workshops to concerts and festivals. With an intuitive interface and powerful management tools, EventHive helps organizers showcase their events effectively while enabling attendees to find and register for experiences that match their interests. Whether you're hosting a corporate conference or looking for local entertainment, EventHive simplifies event discovery and management, creating meaningful connections between organizers and participants.

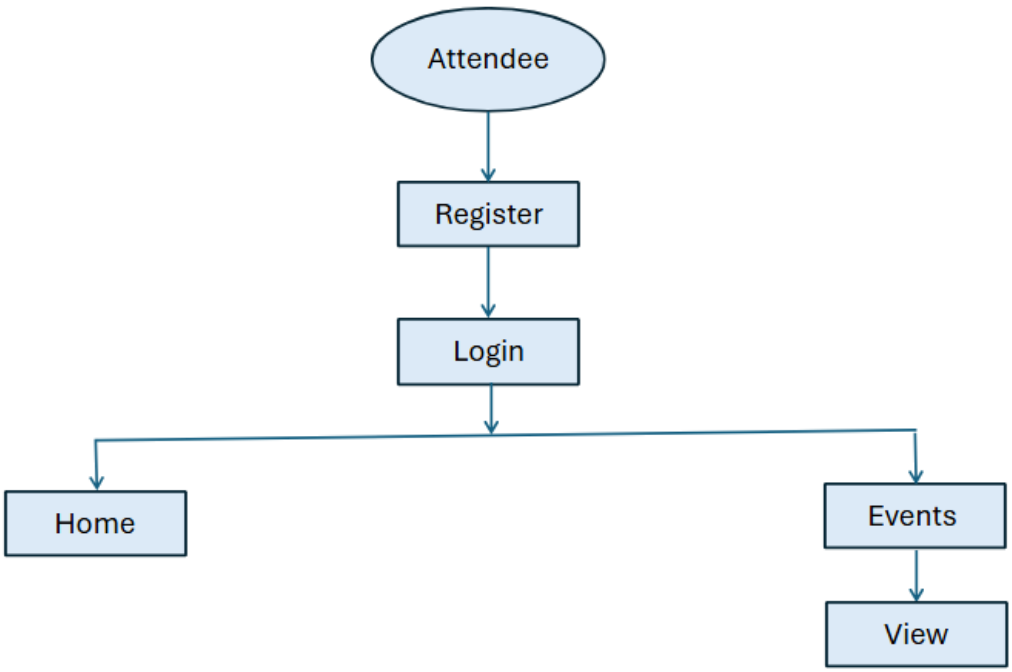
Users of the System:

1. **Organizer**
2. **Attendee**

Organizer Flow Diagram:



Attendee Flow Diagram:



Organizer Actions

Create Events: Organizers create new events by providing the title, category, ticket price, description, and event images.

Attendee Actions

View Events: Attendees can browse available events and view event details.

Modules of the Application:

Organizer:

1. Register
2. Login
3. Home
4. Events
    1. Add Event
    2. View Events
    3. Edit Event
    4. Delete Event

Attendee:

1. Register
2. Login
3. Home
4. Events

1. View Events

Technology Stack

Front End

- React

Back End

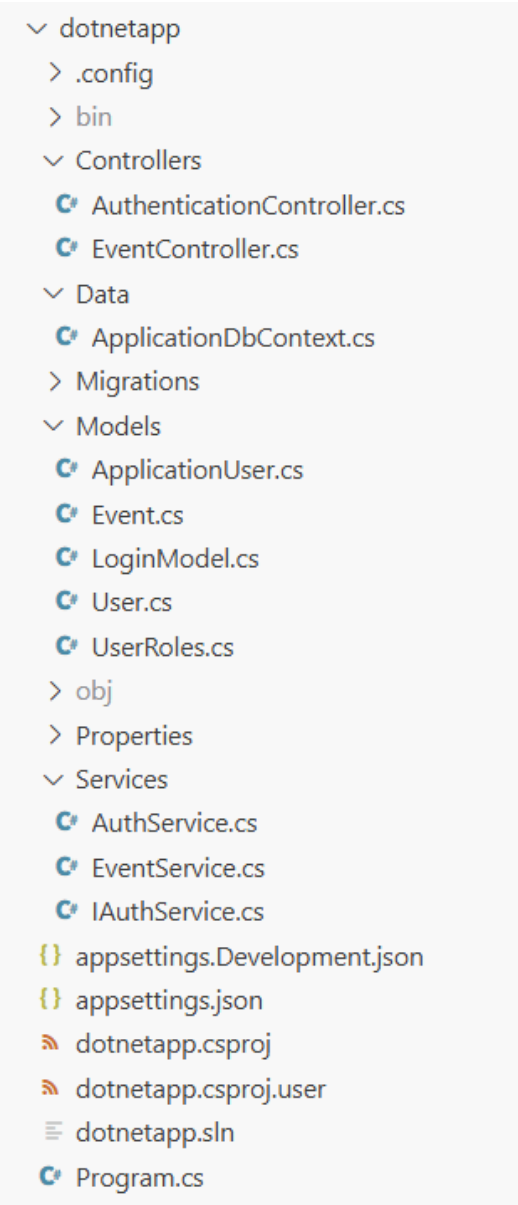
- .NET Web API, EF Core, Microsoft SQL Server Database.

Application assumptions:

1. The login page should be the first page rendered when the application loads.
2. Manual routing should be restricted by implementing Auth Guard, utilizing the canActivate interface. For example, if the user enters as http://localhost:8080/dashboard or http://localhost:8080/user the page should not navigate to the corresponding page instead it should redirect to the login page.
3. Unless logged into the system, the user cannot navigate to any other pages.
4. Logging out must again redirect to the login page.

Backend Requirements:

Create the required folders and files inside the `dotnetapp` folder according to the folder structure shown in the screenshot.



<u>ApplicationDbContext</u>: (**/Data/ApplicationDbContext.cs**)
Namespace: **dotnetapp.Data**
Inside **Data** folder, create **ApplicationDbContext** file with the following **DbSet** mentioned below.

```
public DbSet<Event> Events { get; set; }
public  DbSet<User> Users { get; set; }
```

<u>Model Classes:</u>

Inside **Models** folder create all the model classes mentioned below.
Namespace: All the model classes should locate within the **dotnetapp.Models** namespace.

User (Models / User.cs):
This class stores the user role (**Organizer /Attendee**) and all user information.

Properties:

- UserId: int
- Email: string
- Password: string
- Username: string
- MobileNumber: string
- UserRole: string (**Organizer /Attendee**)

Event (Models/Event.cs):

This class represents adding events by Organizer.

Properties:

- EventId: int
- Title: string
- Category: string
- TicketPrice: decimal
- EventImage: string
- Description: string

LoginModel (Models / LoginModel.cs):
This class stores the email and password to authenticate the user during login.
Properties:
- Email: string
- Password: string

UserRoles (Models / UserRoles.cs):
This class defines constants for user roles.
Constants:

1. Organizer: string
2. Attendee: string

ApplicationUser (Models / ApplicationUser.cs):
This class represents a user in the application, inheriting from **IdentityUser** class.
Property:
- Name: string (Max length 30)

Services:
Inside "**Services"** folder, create all the services file mentioned below.

Namespace: All the services files should locate within the **dotnetapp.Services** namespace.

EventService (Services/EventService.cs):

The EventService class provides CRUD operations (Create, Read, Update, Delete) for managing event entities in the application, ensuring proper handling and validation of event data within the context of an ApplicationDbContext. It simplifies and abstracts the interaction with the database, offering a centralized service for event management.

Constructor:

```
public EventService(ApplicationDbContext context)
        {
            _context = context;
        }
```

Functions:

1. **public async Task<IEnumerable<Event>> GetAllEvents():**
    - Retrieves all events from the database.
2. **public async Task<Event> GetEventById(int eventId):**
    - Retrieves an Event from the database with the specified EventId.
3. **public async Task<bool> AddEvent(Event eventObj):**
    - Adds a new event to the database.
    - Saves changes asynchronously to the database.
    - Returns true for a successful insertion.
4. **public async Task<bool> UpdateEvent(int eventId, Event eventObj):**
    - Updates an existing Event in the database with the values from the provided event object.
    - If no event with the specified EventId is found, returns false.
    - Saves changes asynchronously to the database.
    - Returns true for a successful update.
5. **public async Task<bool> DeleteEvent(int eventId):**
    - Deletes an event from the database with the specified EventId.
    - Retrieves the event based on the provided EventId.
    - If no event with the specified EventId is found, returns false.
    - If the event is found, deletes it from the database.

- Saves changes asynchronously to the database.
- Returns true for a successful deletion.

<u>AuthService</u> (Services / AuthService.cs):

The **AuthService** class is responsible for user authentication and authorization.

<u>Constructor:</u>

public **AuthService**(UserManager<ApplicationUser> userManager, RoleManager<IdentityRole> roleManager, IConfiguration configuration, ApplicationDbContext context)

{

this.userManager = userManager;

this.roleManager = roleManager;

_configuration = configuration;

_context = context;

}

Functions:

1. public async Task<(int, string)> Registration (User model, string role):

- Check if the email already exists in the database. If so return "**User already exists**".
- Registers a new user with the provided details and assigns a role.
- If any error occurs return "**User creation failed! Please check user details and try again**".
- Return "**User created successfully!"** for the successful register.

2. public async Task<(int, object)> Login (LoginModel model):

- Find user by email in the database.
- Check if user exists, if not return “**Invalid email**”.
- If the user exists, check the password is correct, if not return “**Invalid password**”.
- Logs in a user with the provided credentials and generates a **JWT** token for authentication.

3. private string GenerateToken(IEnumerable<Claim> claims):

- Generates a JWT token based on the provided claims.

<u>IAuthService</u> (Services / IAuthService.cs):

The **IAuthService** is an interface that defines methods for user registration and login.

Methods:

1. Task< (int, string)> Registration (User model, string role);
2. Task< (int, object)> Login (LoginModel model);

<u>Controllers</u>:

Inside "**Controllers"** folder creates all the controllers file mentioned below.

Namespace: All the controllers files should locate within the **dotnetapp.Controllers** namespace.

AuthenticationController (Controllers / AuthenticationController.cs):

This controller handles user authentication and registration requests.

Functions:

1. public async Task<IActionResult> Login(LoginModel model)

a. Accepts login requests, validates the payload, and calls the authentication service to perform user login.

b. It utilizes **_authService.Login(model)** method.

c. Returns a **200 OK response** with a JWT token upon successful login.

d. If an exception occurs during the process, it returns a **500 Internal Server Error** response with the exception message.

2. public async Task<IActionResult> Register(User model):

a. Accepts registration requests, validates the payload. If fails, then returns error.

b. Calls the authentication service to register a new user(**_authService.Registration(model, model.UserRole)**). Returns a **200 OK response with** success message upon successful registration.

c. If an exception occurs during the process, it returns a **500 Internal Server Error** response with the exception message.

EventController (Controllers/EventController.cs):

This controller manages Events, interacting with the **EventService** to perform CRUD operations.

Functions:

1. public async Task<ActionResult<IEnumerable<Event>>> GetAllEvents():

a) The GetAllEvents method is a controller action responsible for retrieving all events.
b) Requires authorization to access (all authenticated users).
c) Calls the _eventService.GetAllEvents() method to fetch all events from the service layer.
d) Returns a 200 OK response with the retrieved events upon successful retrieval.

2. public async Task<ActionResult<Event>> GetEventById(int eventId):

a) The GetEventById method is a controller action responsible for retrieving an event by its ID.
b) Requires Organizer role authorization.
c) Calls the _eventService.GetEventById(eventId) method to retrieve the event from the service layer.
d) If the event is not found, returns a 404 Not Found response with a message "Cannot find any event".
e) If the event is found, returns a 200 OK response with the event data.

3. public async Task<ActionResult> AddEvent([FromBody] Event eventObj):

a) The AddEvent method is a controller action responsible for adding a new event.
b) Requires Organizer role authorization.
c) Implements the logic inside a try-catch block.
d) Receives the event data in the request body.
e) Tries to add the event using the _eventService.AddEvent(eventObj) with a success message "Event added successfully".
f) If adding the event fails, returns a 500 Internal Server Error response with a failure message "Failed to add event".
g) If an exception occurs during the process, returns a 500 Internal Server Error response with the exception message.

4.public async Task<ActionResult> UpdateEvent(int eventId, [FromBody] Event eventObj):

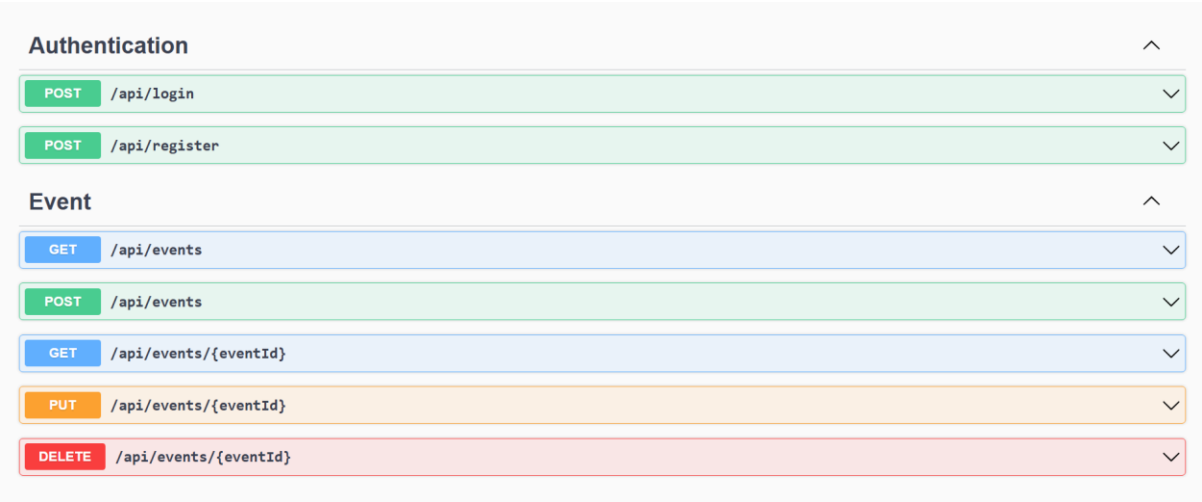
a) The UpdateEvent method is a controller action responsible for updating an existing event.
b) Requires Organizer role authorization.
c) Implements the logic inside a try-catch block.
d) Receives the event ID and updated event data in the request body.
e) Tries to update the event using the _eventService.UpdateEvent(eventId, eventObj) method.
f) If the update is successful, returns a 200 OK response with a success message "Event updated successfully".

g) If the event is not found, returns a 404 Not Found response with a message "Cannot find any event".
h) If an exception occurs during the process, returns a 500 Internal Server Error response with the exception message.

5. public async Task<ActionResult> DeleteEvent(int eventId):

a) The DeleteEvent method is a controller action responsible for deleting an event.
b) Requires Organizer role authorization.
c) Implements the logic inside a try-catch block.
d) Receives the Event ID to be deleted.
e) Tries to delete the event using the _eventService.DeleteEvent(eventId) method.
f) If the deletion is successful, returns a 200 OK response with a success message "Event deleted successfully".
g) If the event is not found, returns a 404 Not Found response with a message "Cannot find any event".
h) If an exception occurs during the process, returns a 500 Internal Server Error response with the exception message.

Endpoints:



<u>Note:</u>

1. Ensure JWT authentication is applied to all endpoints, with role-based access control:
   - The endpoint for adding an event requires a valid JWT in the header. If not provided or invalid, return an Unauthorized error.
   - Only users with the "Organizer" role can access this endpoint. If a user with the "Attendee" role (or any other unauthorized role) attempts access, return a Forbidden error.
2. If there is any unused Endpoints, you can use it for a future use (There will be no testcase).

1. <u>Login</u>: [Access for both Attendee and Organizer]

Endpoint name: "/api/login"

Method: POST

Request body: {
"Email": "string",
"Password": "string"
}

Response:

Status Code	Response body

200 (HttpStatusCode OK)	JSON object containing a JWT token.
	Example:
	{
	"token":
	"eyJhbGciOiJIUzI1NiiQWRtaW4iLCJqdGkiOiJmZTUyYzAxZS1",
	"role": "Organizer",
	"userName": "john_doe",
	"userId": 1
	}

400 BadRequest	JSON object containing Error message.
500 Internal Server Error	JSON object containing Error message.

<u>2. Register</u>: [Access for both Attendee and Organizer]

Endpoint name: "/api/register"

Method: POST

Request body:

{
"Username": "string",
"Email": "user@example.com",
"MobileNumber": "9876541221",
"Password": "Pass@2425",
"UserRole": "string"
}

Response:

Status Code	Response body

200 (HttpStatusCode OK)	{
	"Status": "Success",
	"Message": "User created successfully!"
	}

400 BadRequest	JSON object containing Error message.
500 Internal Server Error	JSON object containing Error message.

3. Get all Events: [Access for both Attendee and Organizer]

Endpoint name: "/api/events"

Method: GET

Response:

Status Code	Response body
200 (HttpStatusCode OK)	JSON object containing details of all events.
500 Internal Server Error	JSON object containing Error message.

4. Get Event by ID: [Access for Organizer only]

Endpoint name: "/api/events/{eventId}"

Method: GET

Response:

Status Code	Response body
200 (HttpStatusCode OK)	JSON object containing details of a event.
404 Not Found	JSON object containing Error message. Example: { "message": "Cannot find any event" }
500 Internal Server Error	JSON object containing Error message.

5. Add Event: [Access for Organizer only]

Endpoint name: "/api/ events"

Method: POST

Request body:

```
{
  "EventId": 0,
  "Title": "string",
  "Category": "string",
  "TicketPrice": 0,
  "EventImage": "string",
  "Description": "string"
}
```

Response:

Status Code	Response body

200 (HttpStatusCode OK)	Response body: JSON object containing success message. Example: { "message": "Event added successfully" }
500 Internal Server Error	JSON object containing Error message. Example: { "message": "Failed to add event" }

6. Update Event: [Access for Organizer only]

Endpoint name: "/api/events/{eventId}"

Method: PUT

Parameter: eventId

Request body:

```
{
  "EventId": 0,
  "Title": "string",
  "Category": "string",
  "TicketPrice": 0,
  "EventImage": "string",
  "Description": "string"
}
```

Response:

Status Code	Response body
200 (HttpStatusCode OK)	Response body: JSON object containing success message. Example: { "message": "Event updated successfully" }
404 NotFound	JSON object containing Error message. Example: { "message": "Cannot find any event" }
500 Internal Server Error	JSON object containing Error message.

7. Delete Event: [Access for Organizer only]

Endpoint name: "/api/events/{eventId}"

Method: DELETE

Parameter: eventId

Response:

Status Code	Response body
200 OK	JSON object containing success message. Example: { "message": "Event deleted successfully" }
404 NotFound	JSON object containing Error message. Example: { "message": "Cannot find any event" }
500 Internal Server Error	JSON object containing Error message.

Configure Azure Environment and Application Settings

Step 1: Set Azure Environment Variables

Declare the given Azure Credentials in the cred.sh file.

- AZURE_CLIENT_ID
- AZURE_CLIENT_SECRET
- AZURE_TENANT_ID
- AZURE_SUBSCRIPTION_ID

Set the environment variables for the Azure client by running the following commands:

- cd dotnetapp
- source cred.sh

Step 2: Update **appsettings.json**

In the appsettings.json file, declare the following values to configure Key Vault and JWT settings:

```
{
  "KeyVaultConfiguration": {
    "URL": "replace with your keyvault url",
    "ConnectionStringSecretName": " replace with your connect secret name",
    "BlobSecret": " replace with your blob secret"
  },
  "BlobContainerName": " replace with your blob container name",
  "JWT": {
    "ValidAudience": " replace with your valid audience",
    "ValidIssuer": " replace with your valid issuer",
    "Secret": "replace with your secret"
```

```
    }
}
```

Step 3: Implement **DefaultAzureCredential** in program.cs file

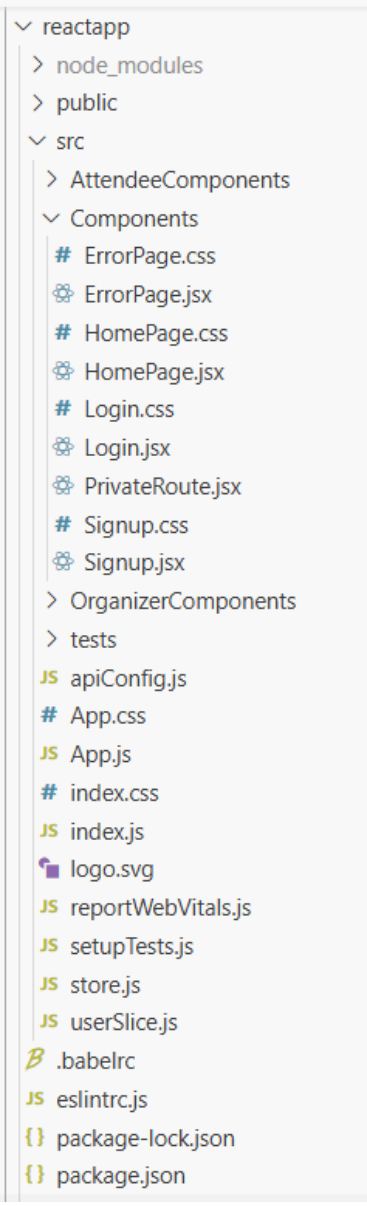
Ensure that the entire application uses the **DefaultAzureCredential** method for authentication.

<u>Frontend Requirements:</u>

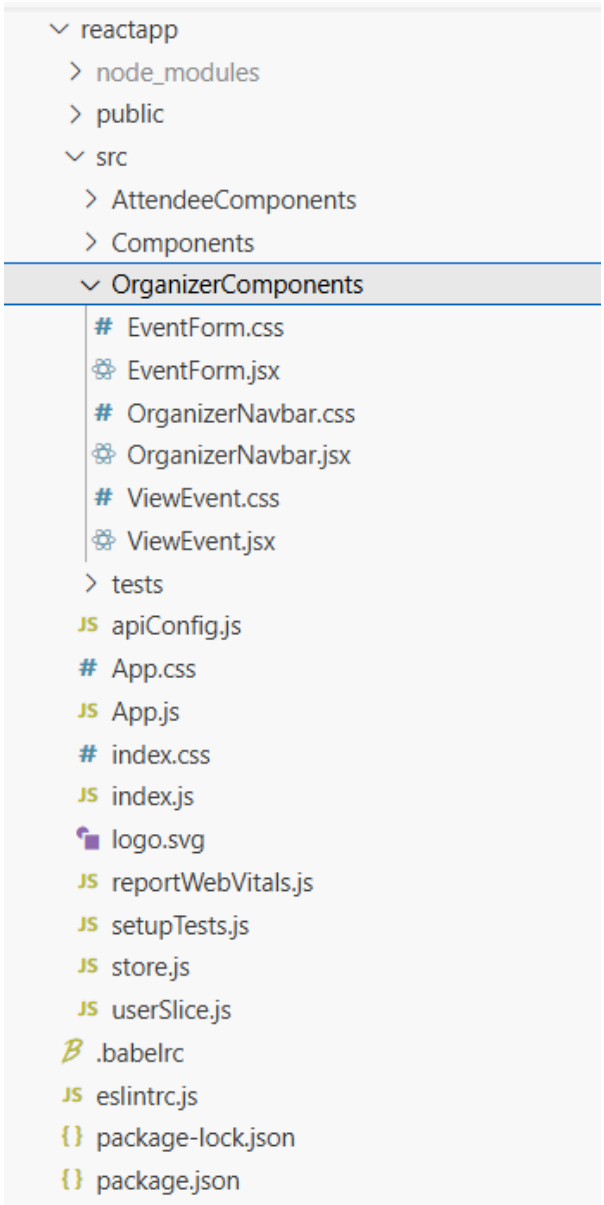
- Create a folder named `Components` inside the `src` to store all common components. (Refer project structure screenshots).
- Create a folder named `OrganizerComponents` inside the `src` to store all components for Organizer. (Refer project structure screenshots).
- Create a folder named `AttendeeComponents` inside the `src` to store all components for Attendee. (Refer project structure screenshots).
- You can create your own components based on the application requirements.

Project Folder Screenshot:

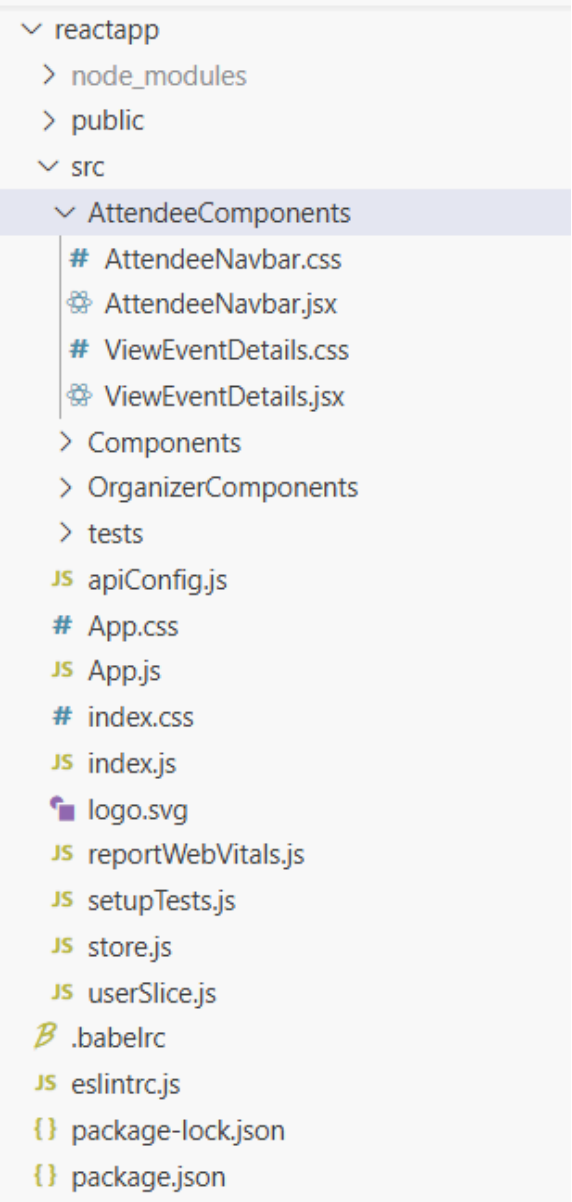
Components:



OrganizerComponents:


reactapp
node_modules
public
src
AttendeeComponents
Components
OrganizerComponents
EventForm.css
EventForm.jsx
OrganizerNavbar.css
OrganizerNavbar.jsx
ViewEvent.css
ViewEvent.jsx
tests
apiConfig.js
App.css
App.js
index.css
index.js
logo.svg
reportWebVitals.js
setupTests.js
store.js
userSlice.js
.babelrc
eslintrc.js
package-lock.json
package.json

AttendeeComponents:



Validations:

Client-Side Validation:

- Implement client-side validation using HTML5 attributes and JavaScript to validate user input before making API requests.
- Provide immediate feedback to users for invalid input, such as displaying error messages near the input fields.

Server-Side Validation:

- Implement server-side validation in the controllers to ensure data integrity.

- Validate user input and API responses to prevent unexpected or malicious data from affecting the application.
- Return appropriate validation error messages to the user interface for any validation failures.

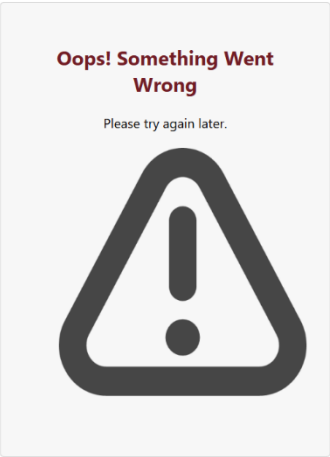
Exception Handling:

- Implement exception handling mechanisms in the controllers to gracefully handle errors and exceptions. Define custom exception classes for different error scenarios, such as API communication errors or database errors.
- Log exceptions for debugging purposes while presenting user-friendly error messages to users. Record all the exceptions and errors handled store in separate table "ErrorLogs".

Error Pages:

Create custom error pages for different HTTP status codes (e.g., 404 Not Found, 500 Internal Server Error) to provide a consistent and user-friendly error experience. Ensure that error pages contain helpful information and guidance for users.

Thus, create a reliable and user-friendly web application that not only meets user expectations but also provides a robust and secure experience, even when faced with unexpected situations. Error page must be displayed if something goes wrong.



Note: Refer to the screenshot for additional details and design the forms accordingly.

Frontend Screenshots:

• All the asterisk (*) marked fields are mandatory in the form. Make sure to mark all the field names with * symbol followed by the validations. Verify that the labels and placeholders for each field are correctly displayed as per the design.

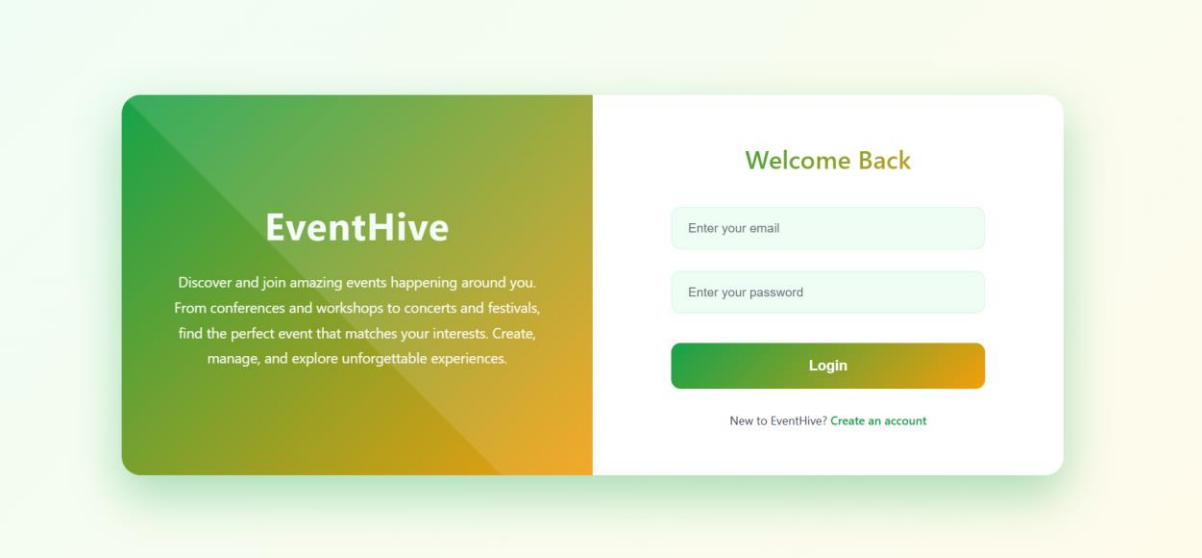
Navigation Bar:

(**OrganizerNavbar component**)

(**AttendeeNavbar component**)

Landing Page:

(**Login component**)



Component features a heading **"EventHive"** accompanied by an introductory message that provides an overview of the application with login form**.**

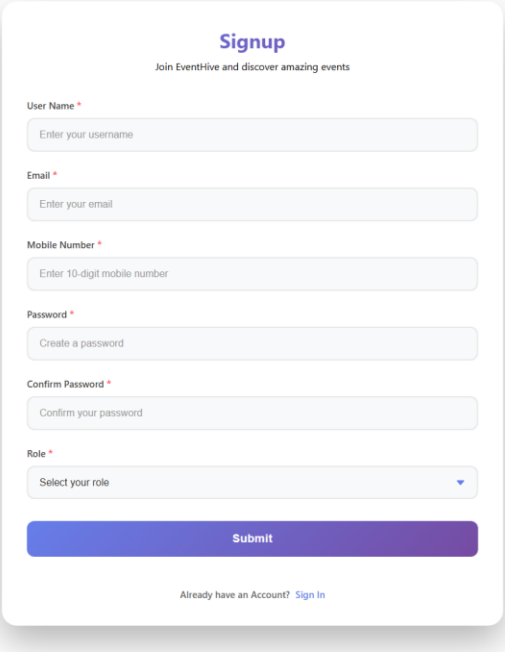
<u>Common Screens (Organizer and Attendee)</u>

Signup Page:

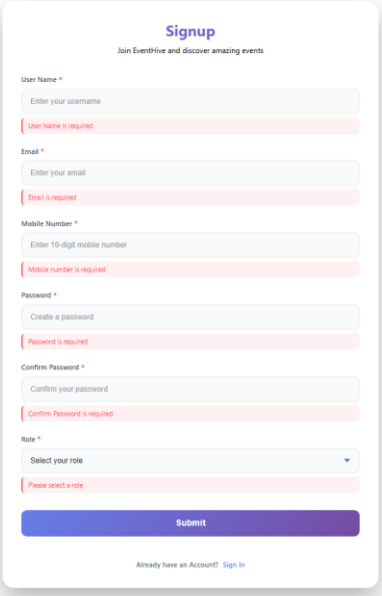
(**Signup component**)

Clicking "**Signup**" in the login page will navigate to the registration page for both roles. Please refer to the screenshots below for validations.

Onclicking “Submit” button with empty field will display the validation message.



Regular expressions (Regex) are also included to validate email formats, mobile numbers, and to ensure password matching.



Signup

Join EventHive and discover amazing events

User Name *

Sam

Email *

sam

Please enter a valid email

Mobile Number *

123456

Mobile number must be 10 digits

Password *

•••

Password must be at least 6 characters

Confirm Password *

•••

Passwords do not match

Role *

Organizer

Submit

Signup

Join EventHive and discover amazing events

User Name *

Sam

Email *

sam@gmail.com

Mobile Number *

1234567890

Password *

•••••••

Confirm Password *

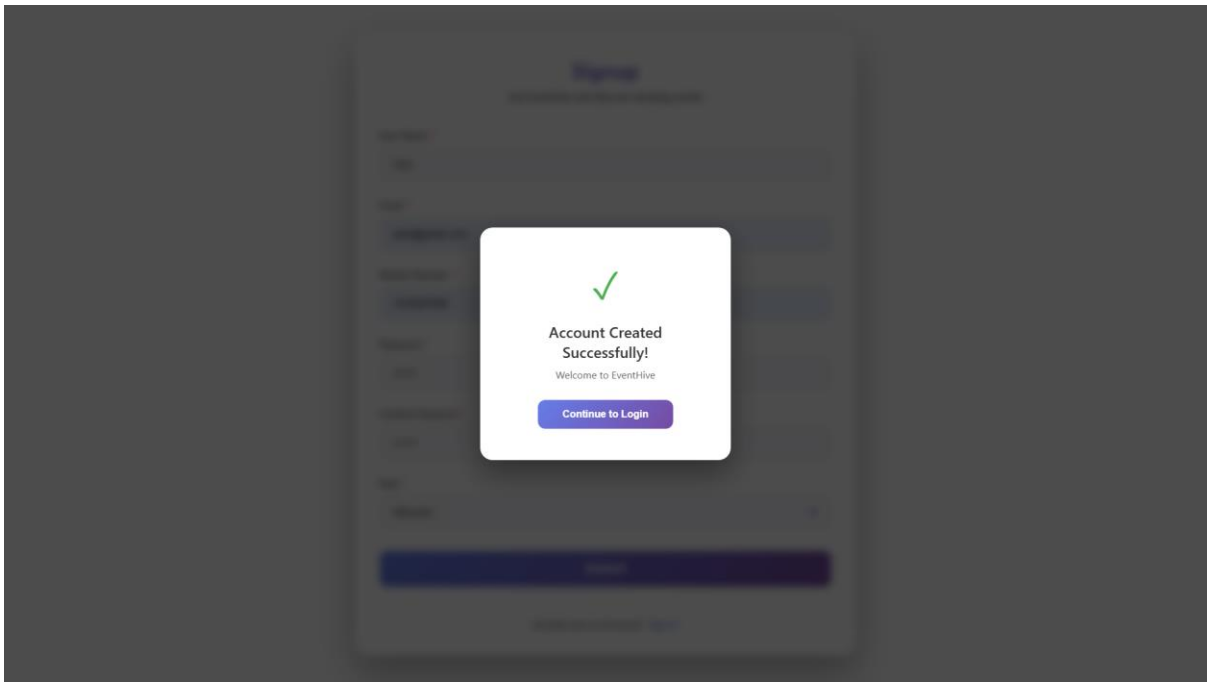
•••••••

Role *

Attendee

Submit

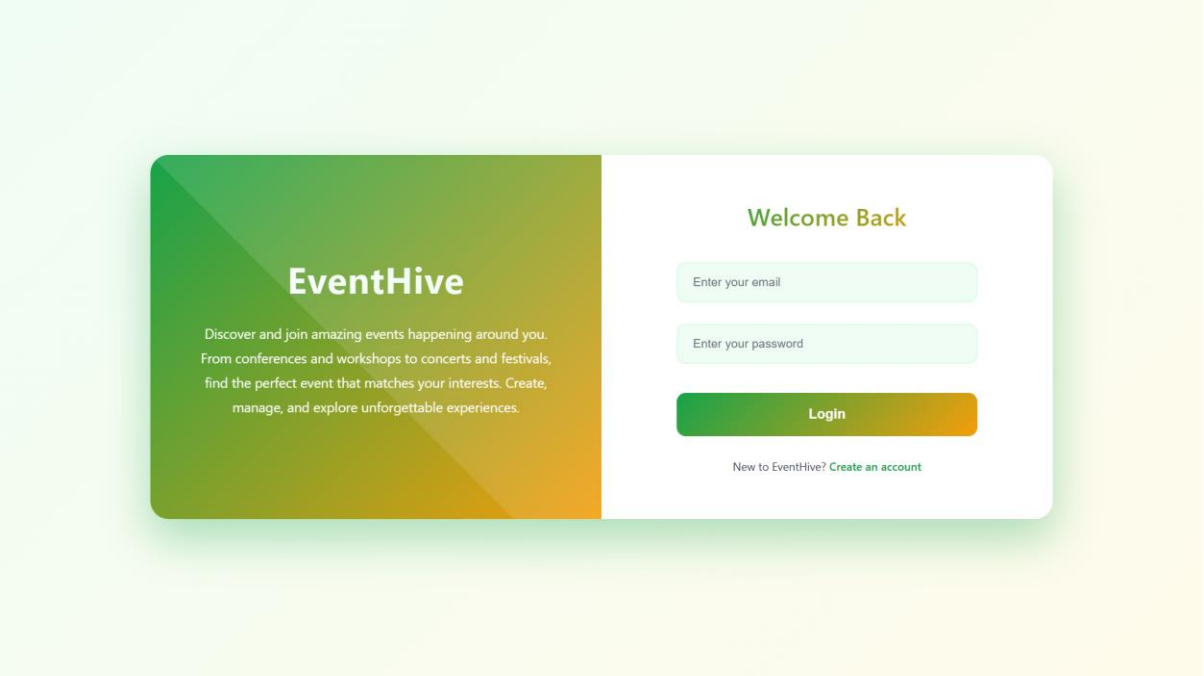
Already have an Account? Sign In



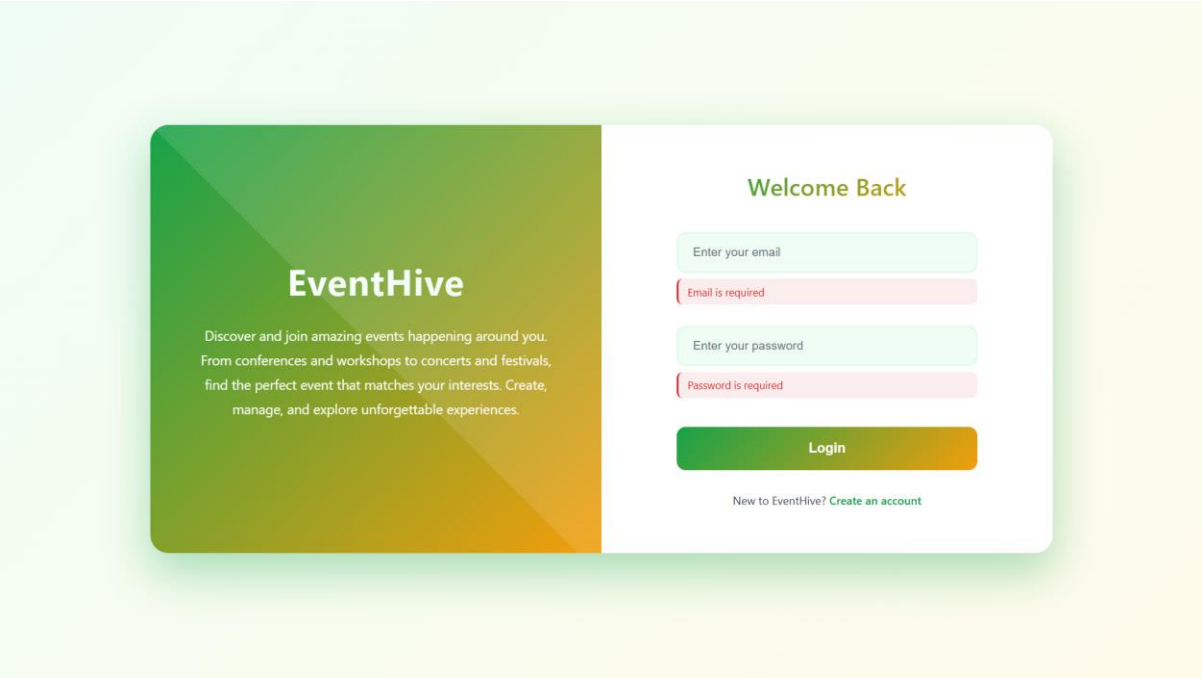
Upon successful completion, a modal will appear, prompting users to click the "Continue to Login" button. Upon clicking this button, users will be directed to the login page, signifying a successful signup process

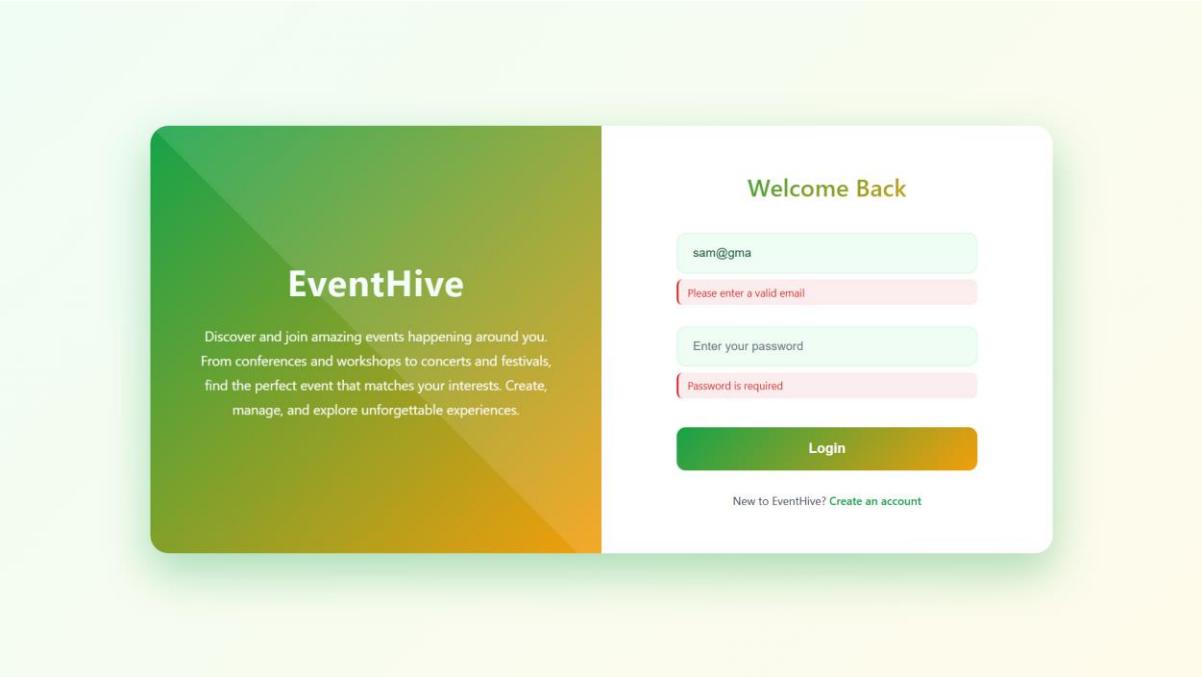
Login Page
(**Login component**)

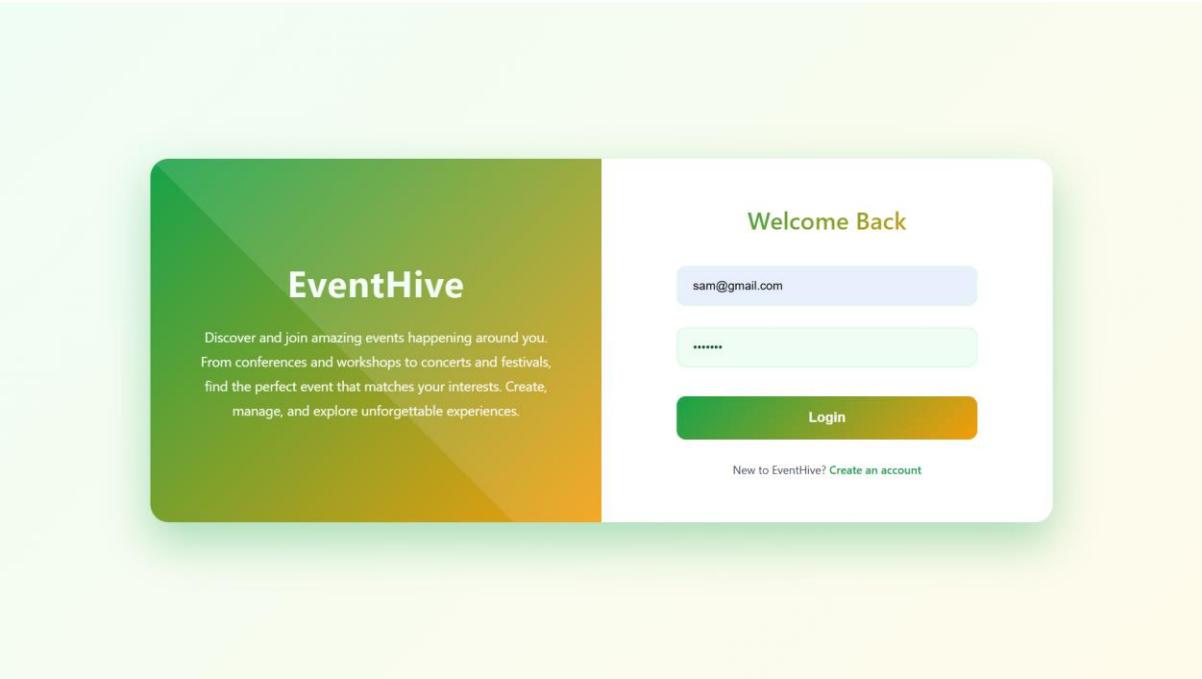
This page is used for logging in to the application. On providing the valid email and password, the user will be logged in.



Perform validations for email and password fields.
On Clicking “Login” button with empty field will display the validation message.







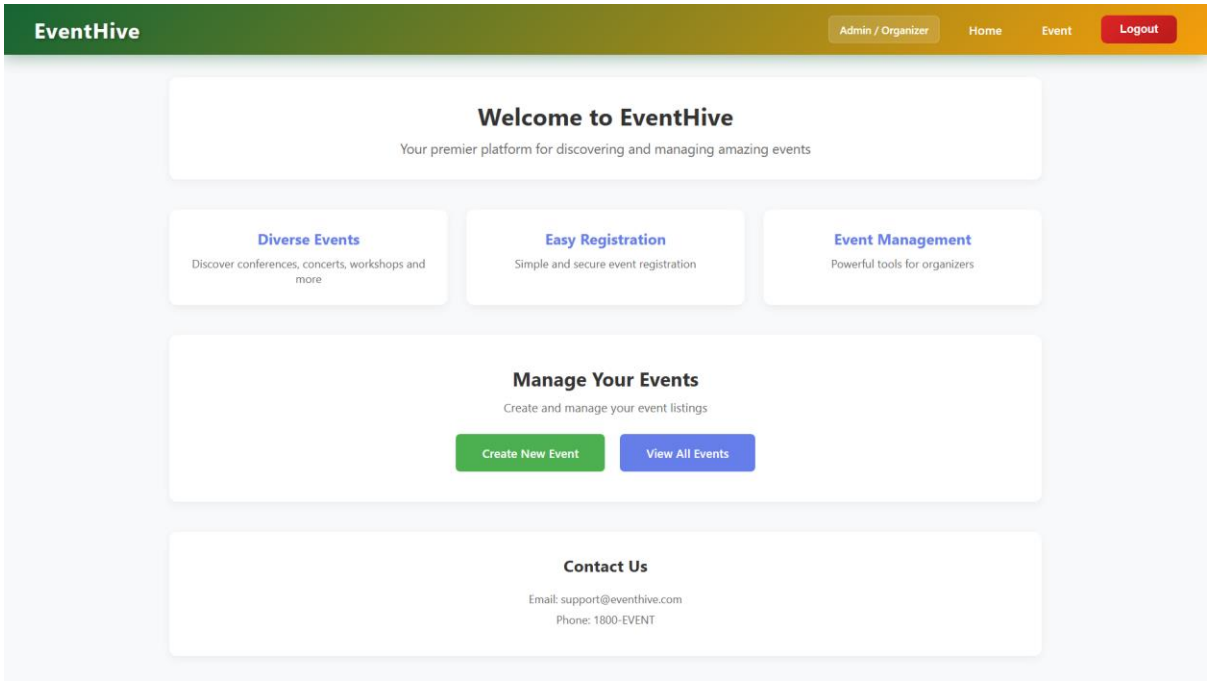
On Clicking the “Login” button, user will be navigated to the (OrganizerNavbar or AttendeeNavbar component) based on their roles.

Organizer Side:

Organizer component:

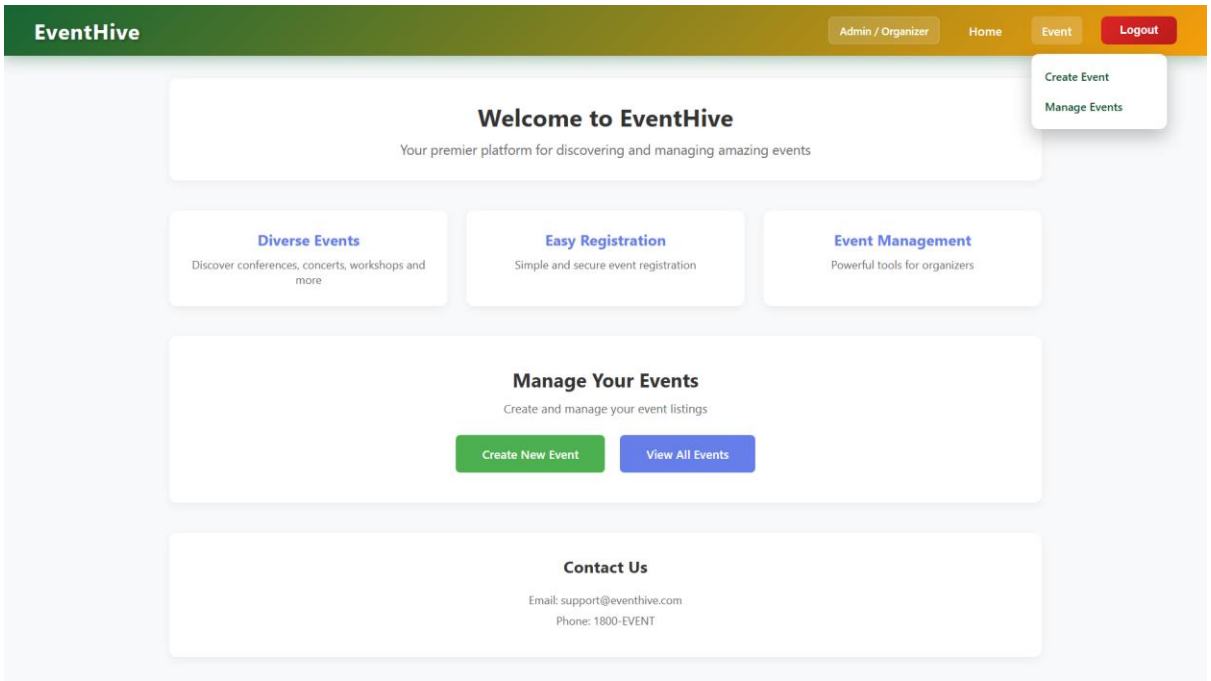
Upon successful login, if the user is a Organizer, the (**OrganizerNavbar** component) will be displayed. If the user is Attendee, the (**AttendeeNavbar** component) will be displayed. Additionally, the role-based navigation bar will also display login information such as the username and role.

HomePage Component: This page is used to display the information about EventHive application. On clicking the ‘**Home**’ tab, user can view the information about the application.

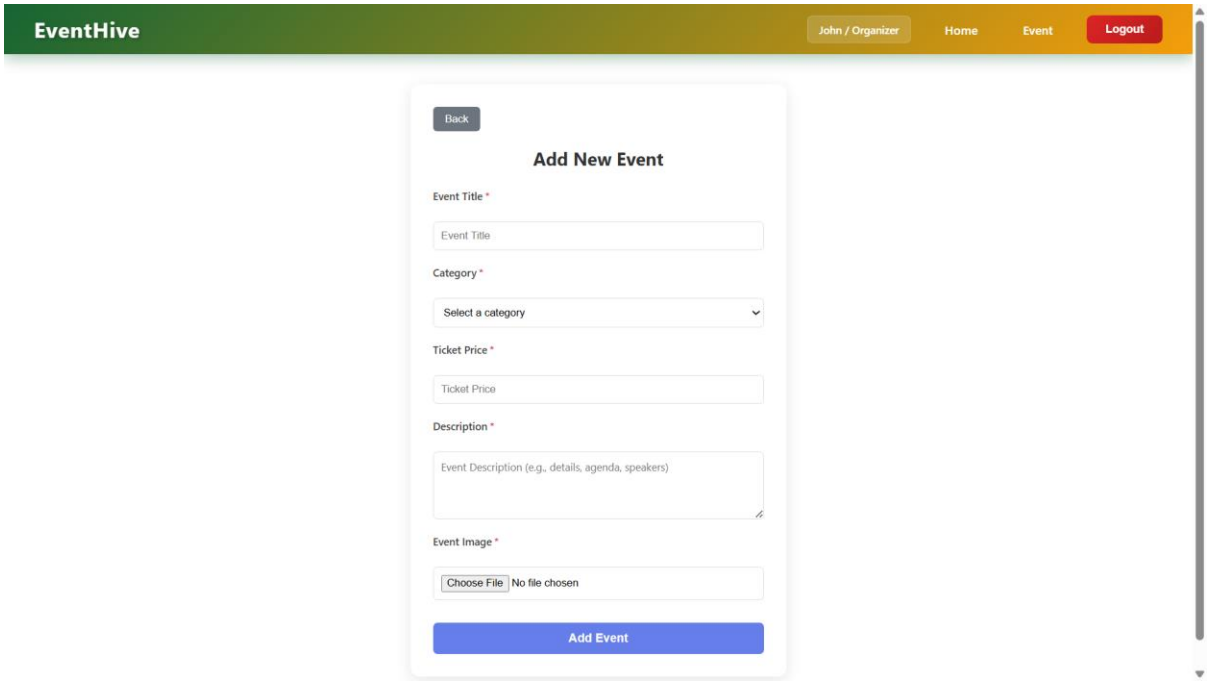


Organizer can navigate to other pages by clicking on the menu available in the navigation bar.

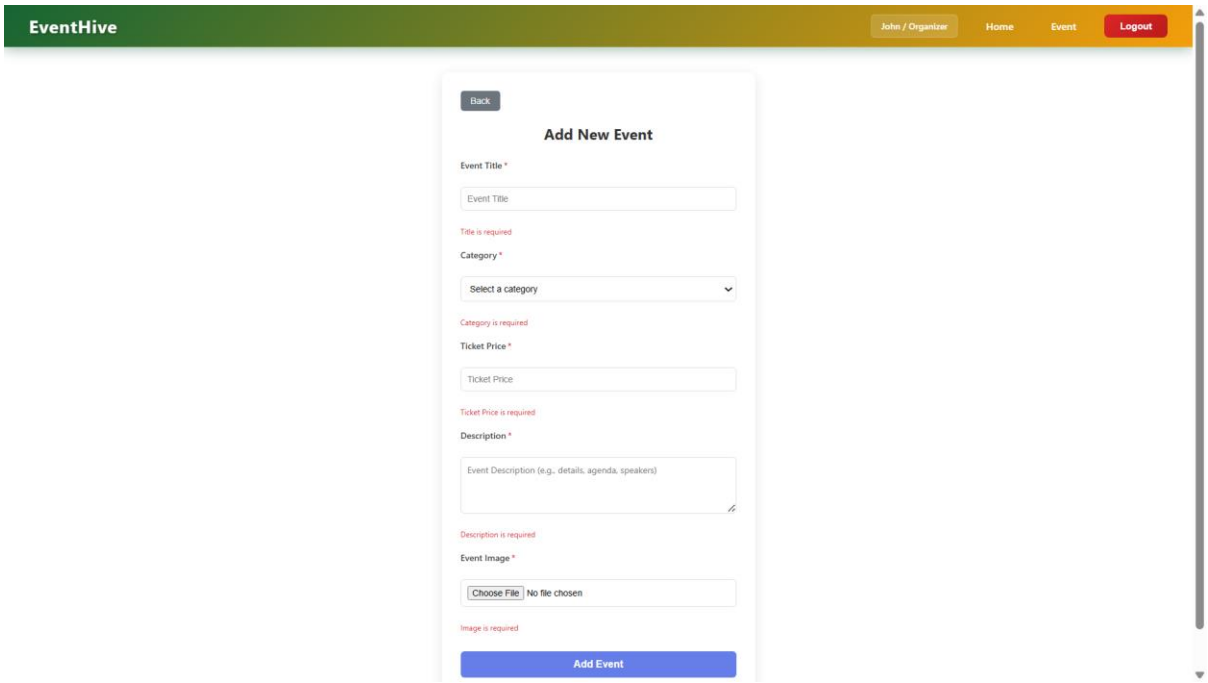
Clicking the “Event” option in the navbar will reveal “Create Event” and “Manage Events” options, allowing the Organizer to add new Events and view existing ones.



By clicking the “Create Event” option will navigate to the “**EventForm**” component, where the Organizer can add Event.



On Clicking “Add Event” button with empty field will display validations error message.



On the 'Add New Event' page, click the 'Category' option to select an event category from a predefined list. This helps to organize and filter event types more effectively.

Back

Add New Event

Event Title *

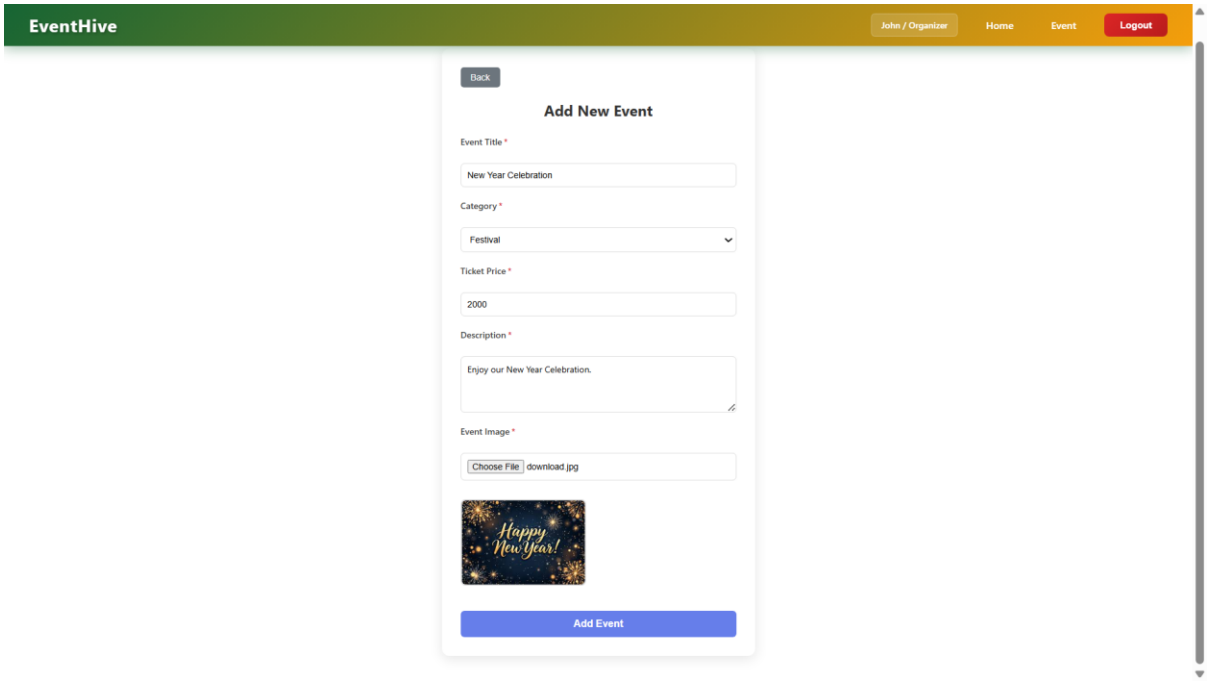
Event Title

Category *

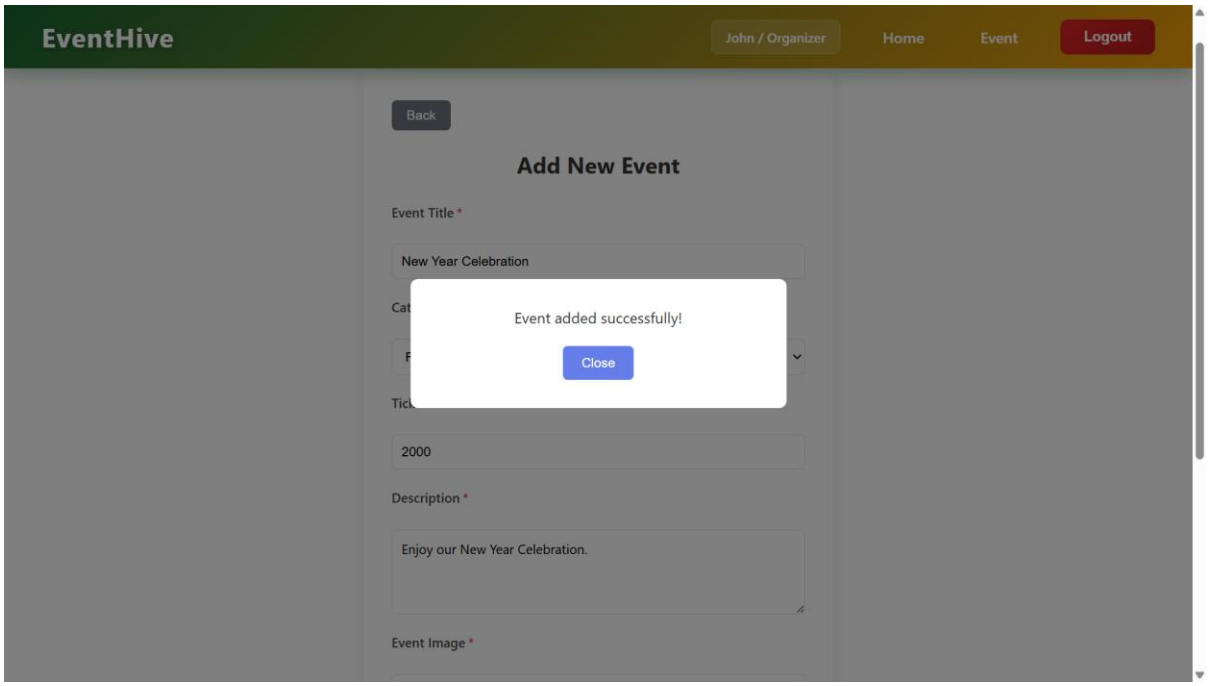
Select a category

- Select a category
- Conference
- Workshop
- Seminar
- Concert
- Festival
- Sports Event
- Networking
- Webinar
- Exhibition
- Charity Event

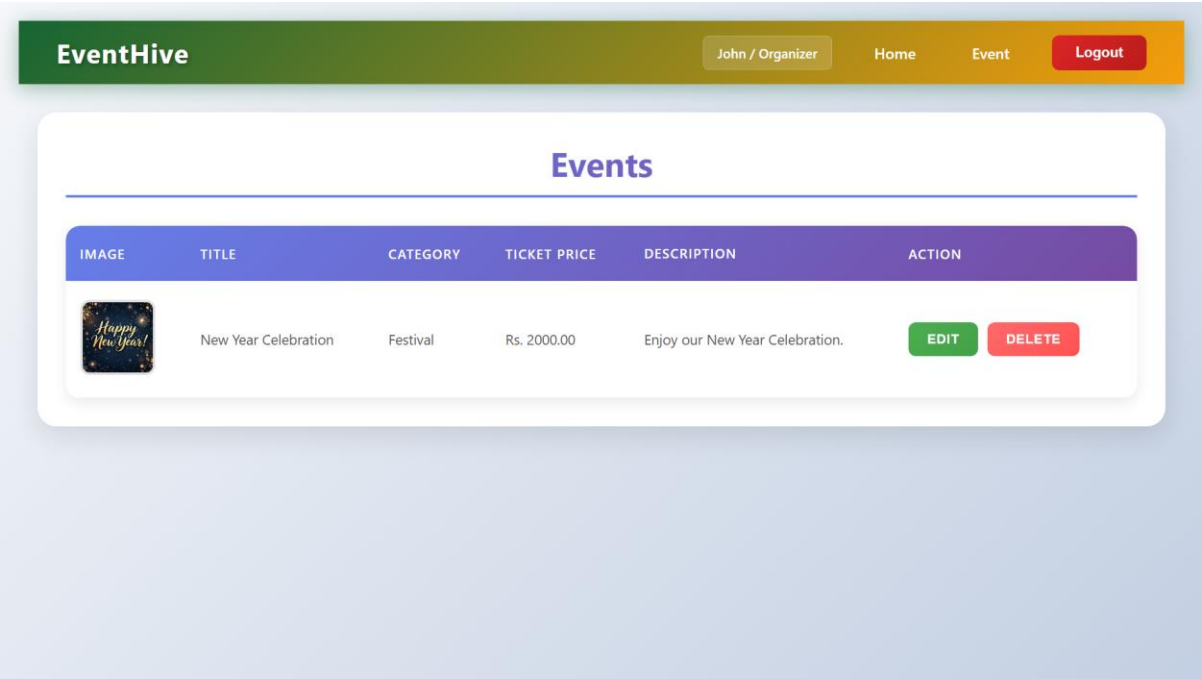
Add Event



Upon successfully adding an Event, a success modal with a "Close" button will be displayed.

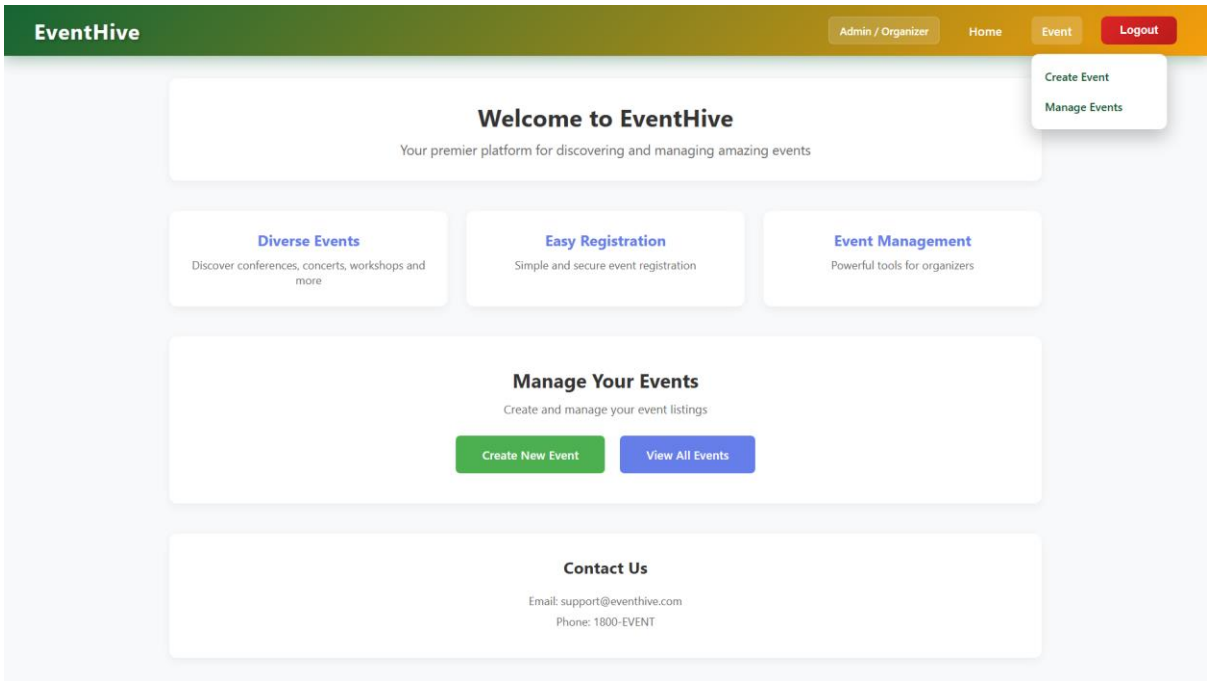


Clicking on “Close” button will navigate to “**ViewEvent**” component

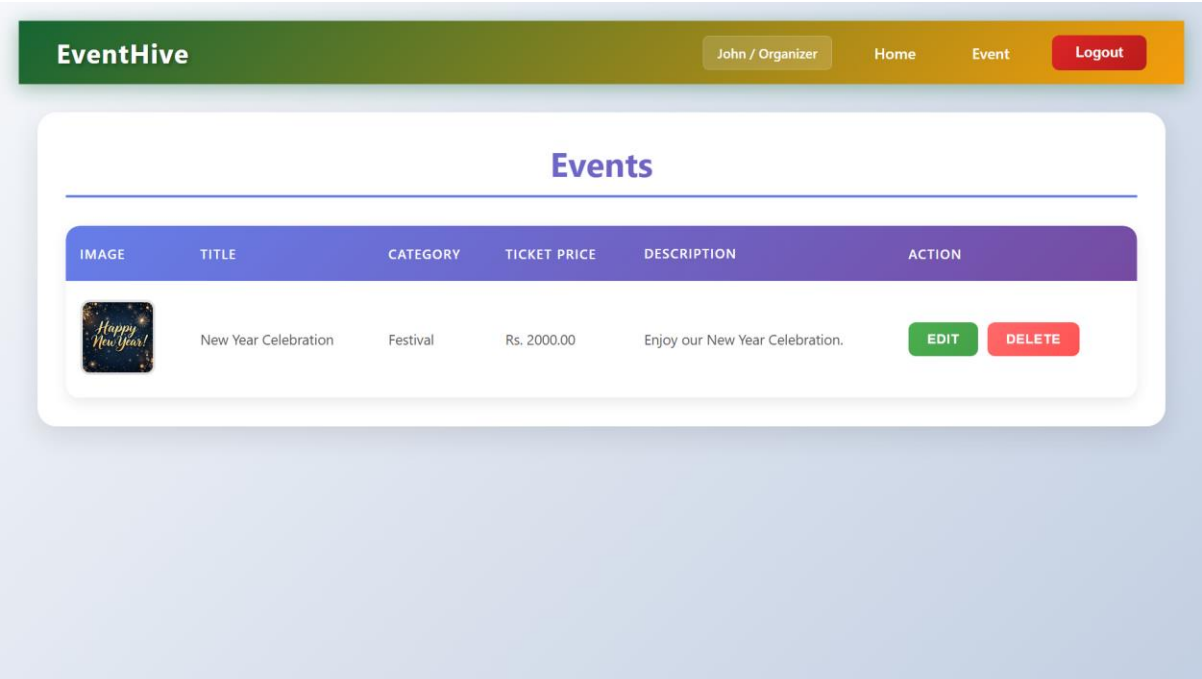


ViewEvent Component:

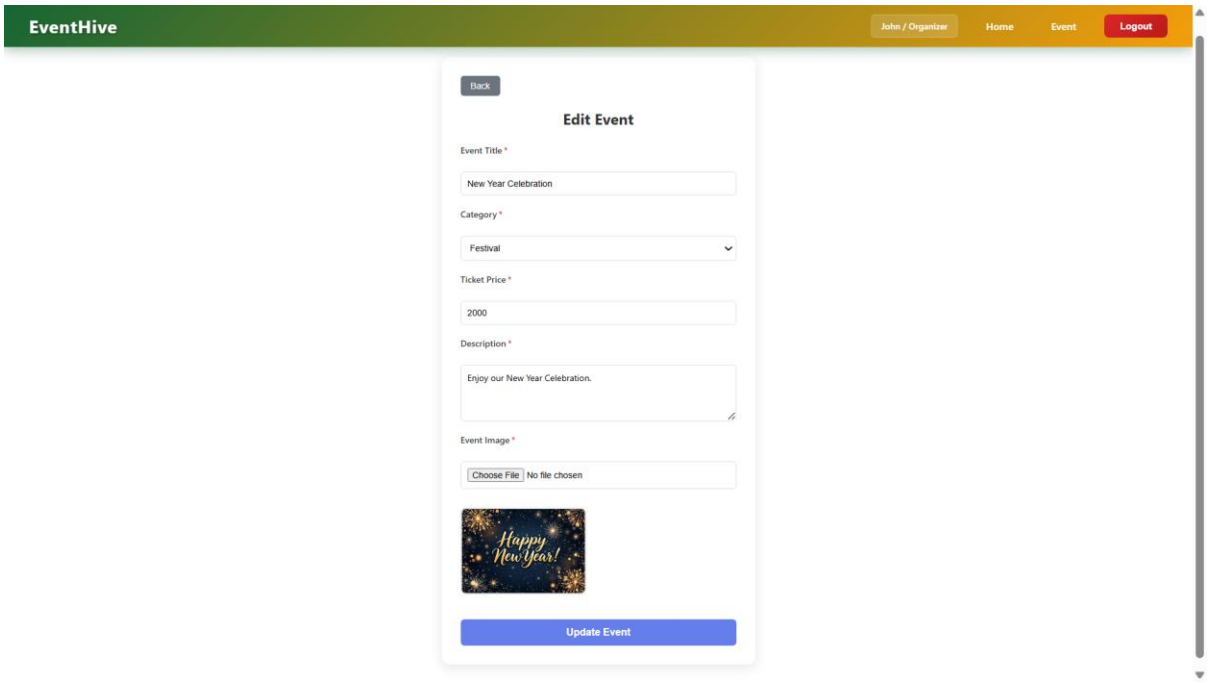
Clicking the “Event” option in the navbar will reveal “Create Event” and “Manage Events” options, allowing the Organizer to add new events and view existing ones.



By clicking the “Manage Events” option will navigate to the “**ViewEvent**” component, where the Organizer can view events.

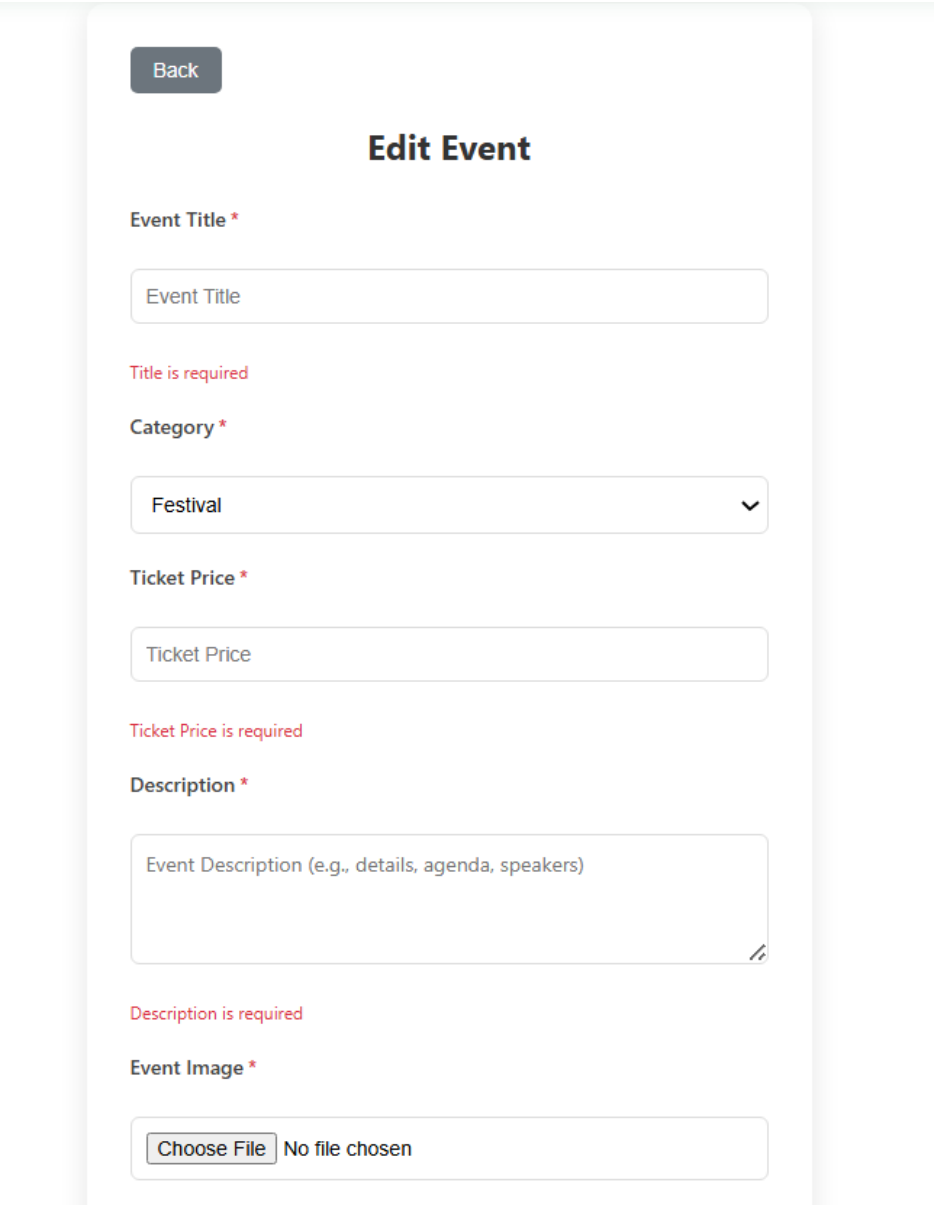


Clicking the "EDIT" button will navigate to the **EventForm** component, displaying the edit form prepopulated with the data of the selected event.

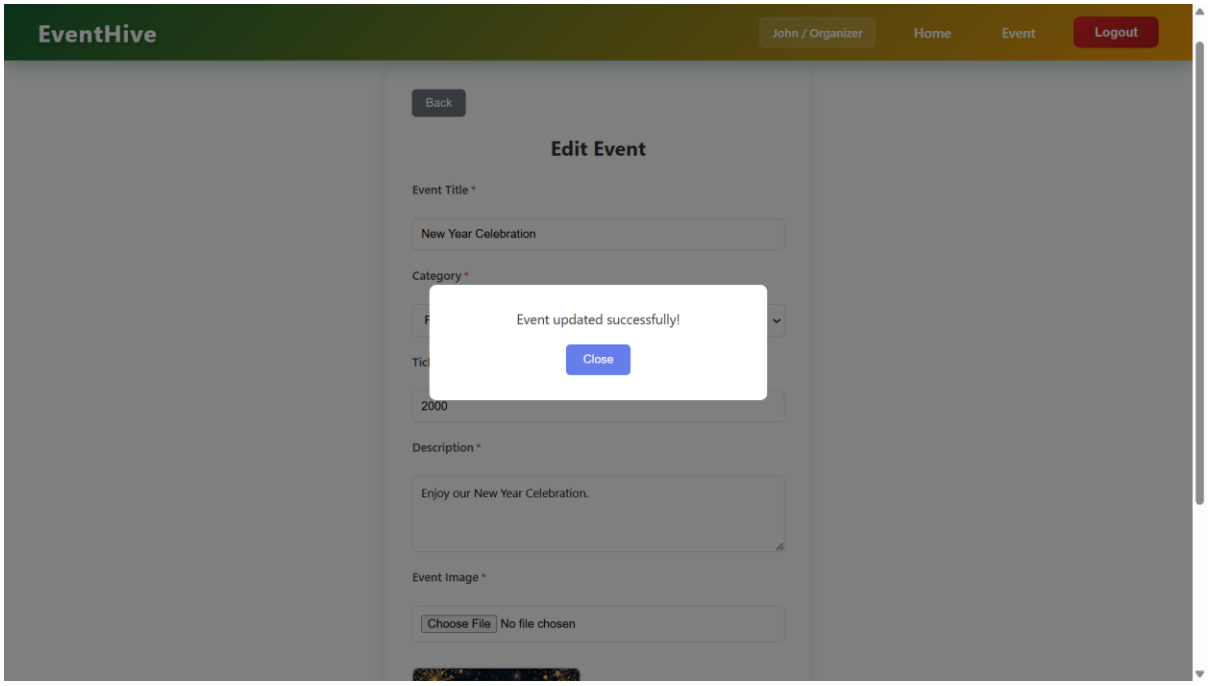


Form includes “Back” button, will navigate to previous page.

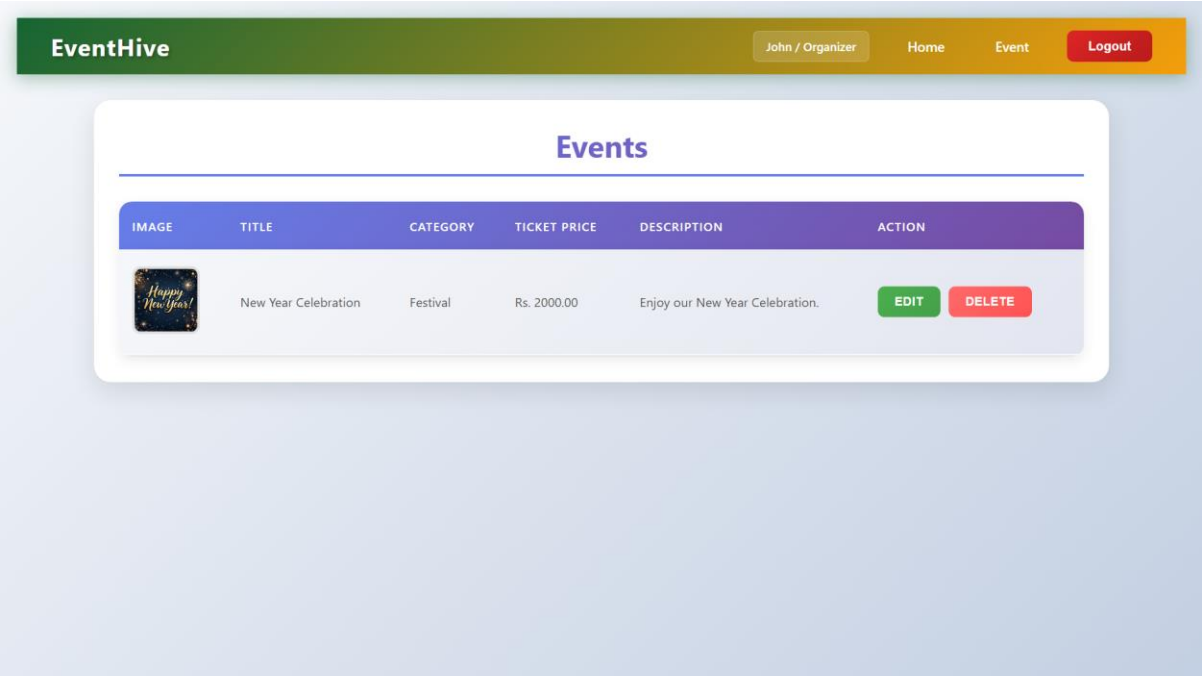
Validation message is displayed on clicking “Update Event” with empty field.



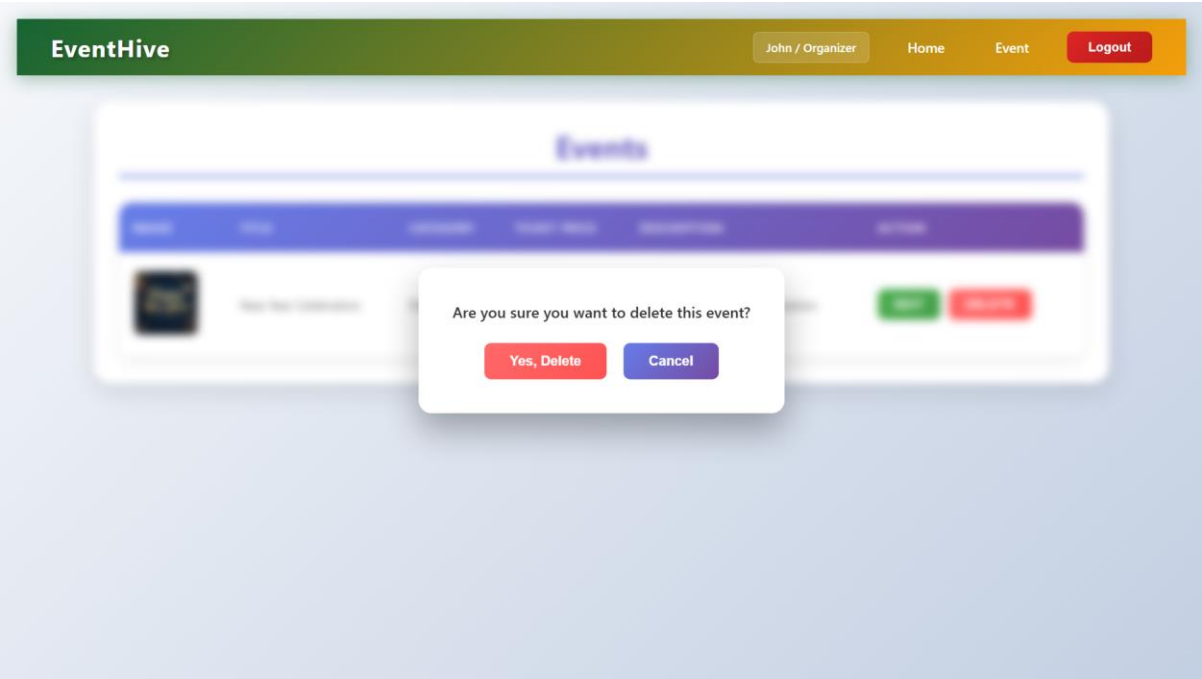
Upon successfully updating an Event, a success modal with a "Close" button will be displayed.



Clicking on “Close” button will navigate to “**ViewEvent**” component



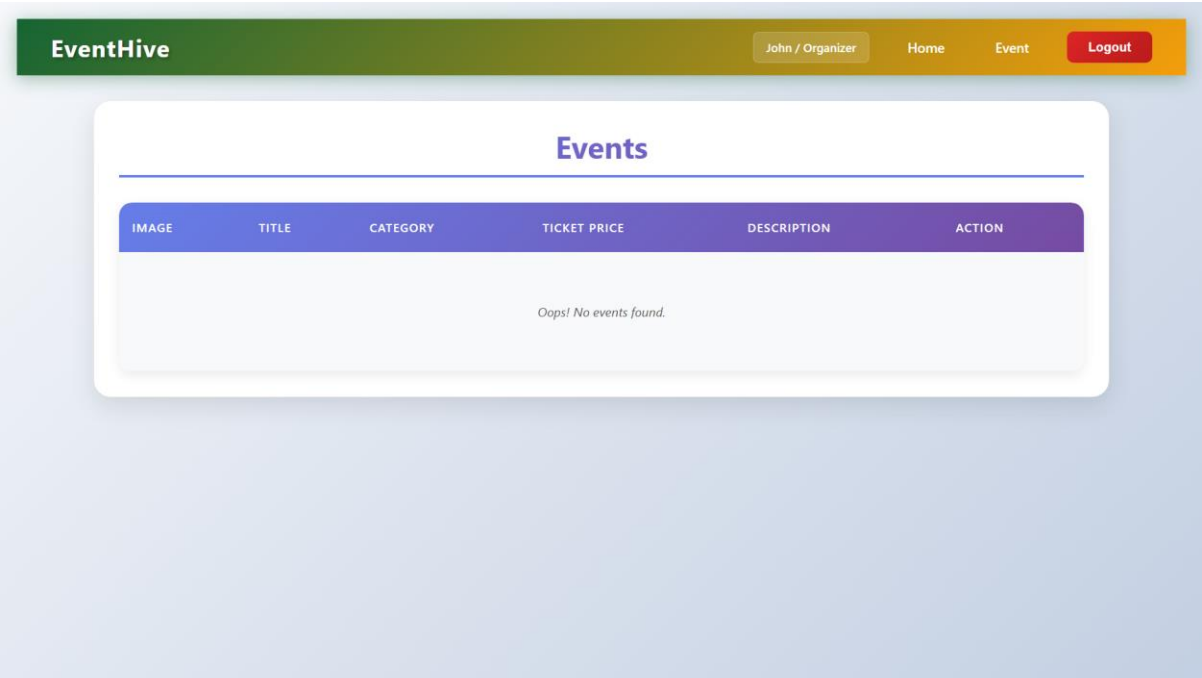
When clicking the "DELETE" button, a confirmation modal will be displayed with the message “**Are you sure you want to delete this event?”**



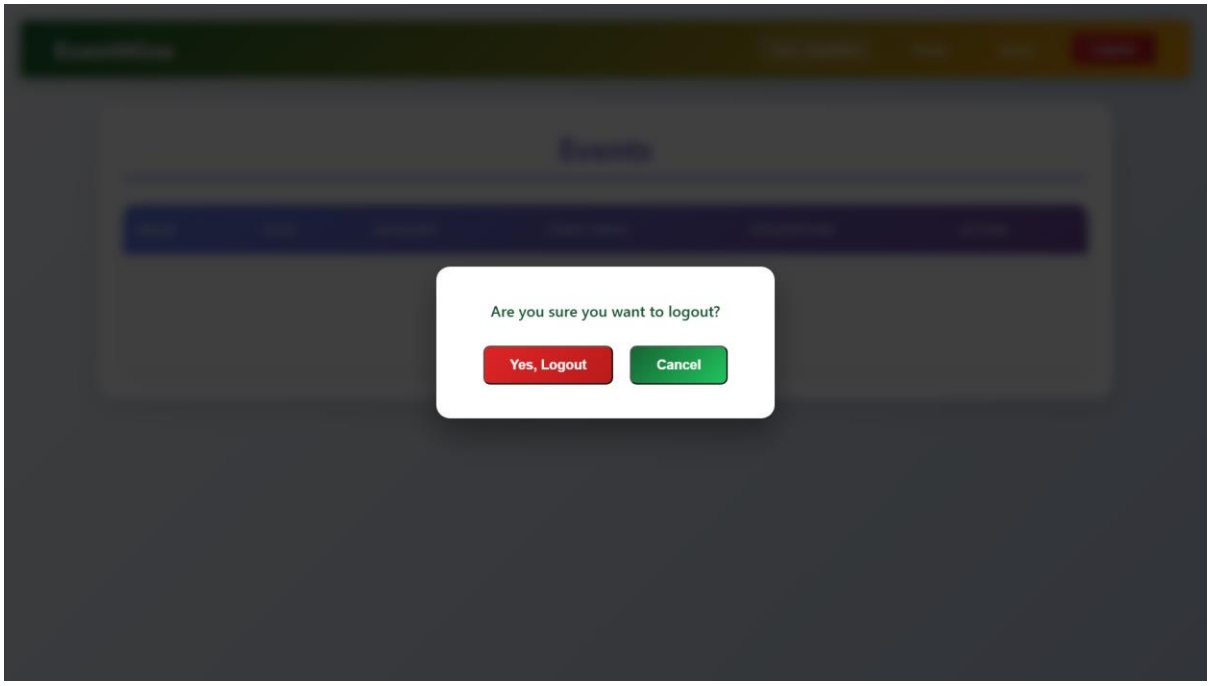
Clicking “Cancel” will close the popup.

Clicking "Yes, Delete" will remove the selected Event and refresh the displayed data.

If data is not available, then “Oops! No events found.” should be displayed.



On clicking the "Logout" button, a pop-up should be displayed with confirmatory message to logout the user.



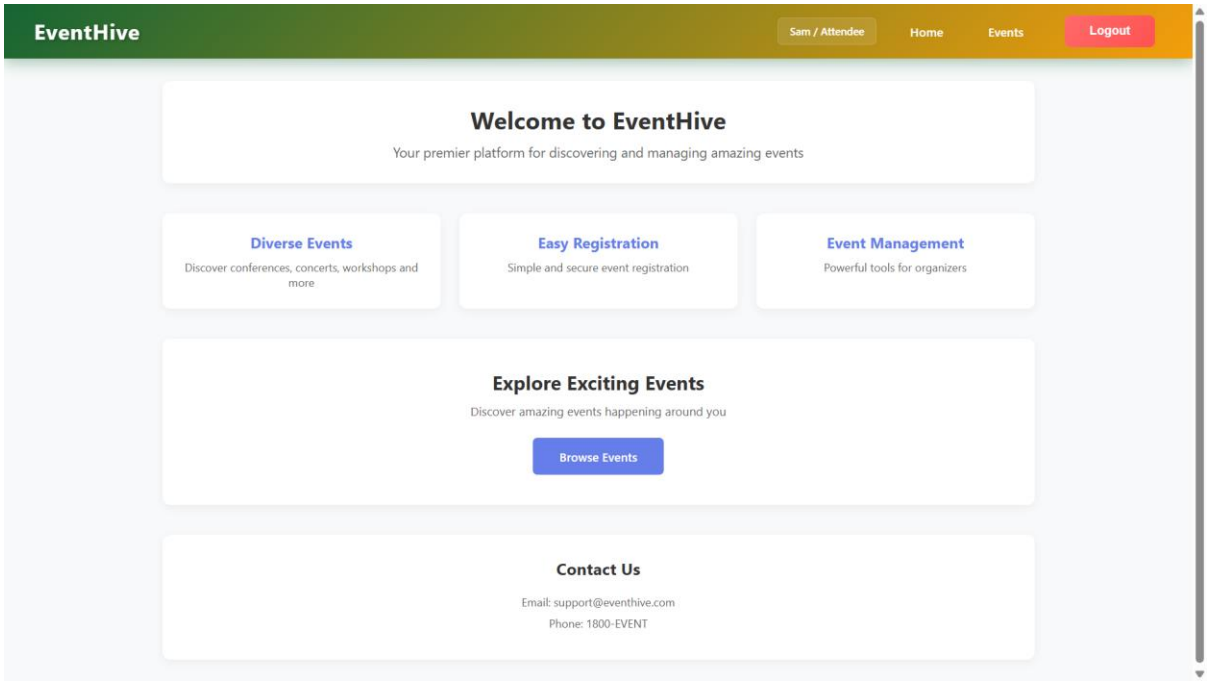
OnClicking "Yes, Logout" will navigate to the login component.
OnClicking “Cancel” will close the modal window displayed.

Attendee Side:

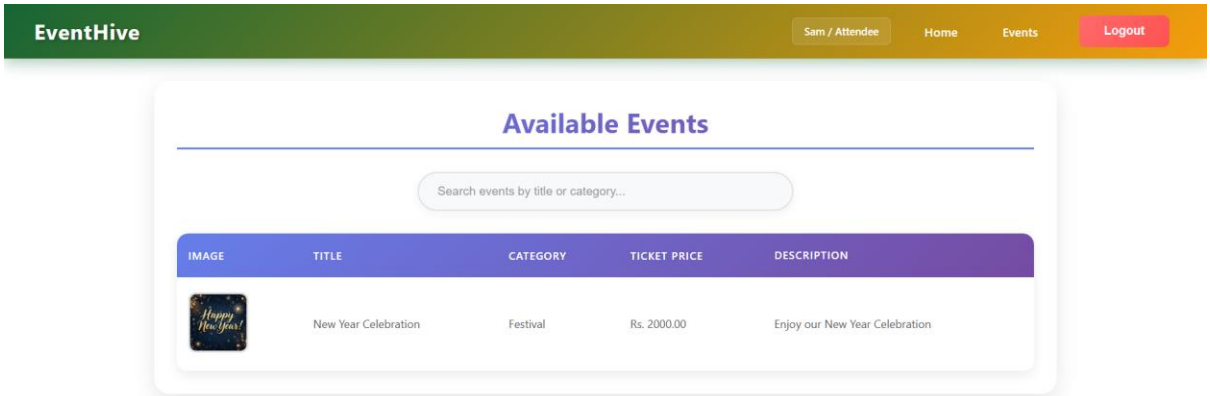
AttendeeNavbar component:

Upon successful login, if the user is an Organizer, the (**OrganizerNavbar** component) will be displayed. If the user is Attendee, the (**AttendeeNavbar** component) will be displayed. Additionally, the role-based navigation bar will also display login information such as the username and role.

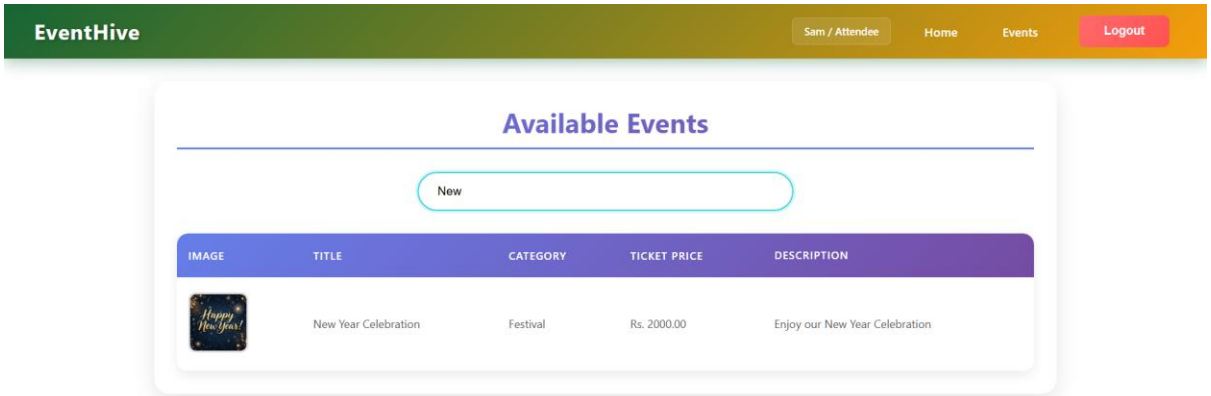
HomePage Component: This page is used to display the information about EventHive application. On clicking the ‘**Home**’ tab, user can view the information about the application.



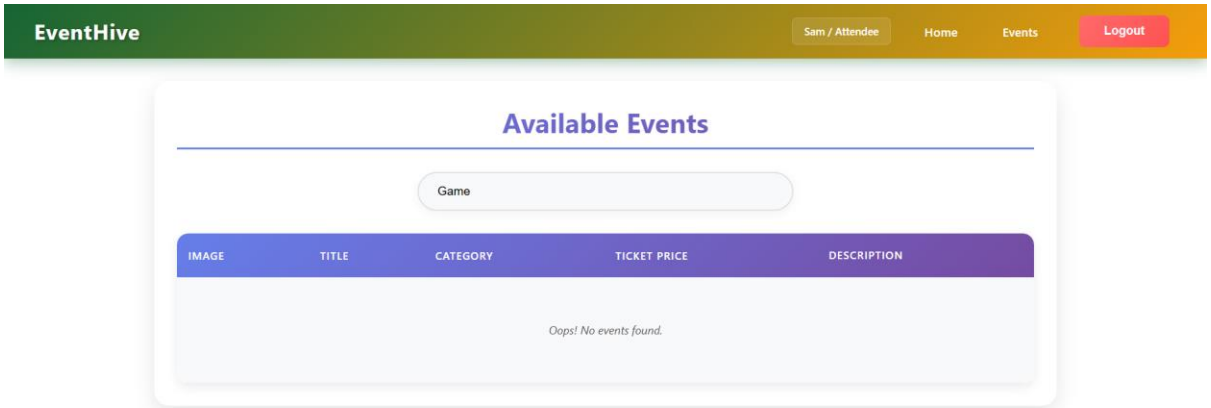
On Clicking over the "Events" item in the navbar, will navigate to “ViewEventDetails” Component



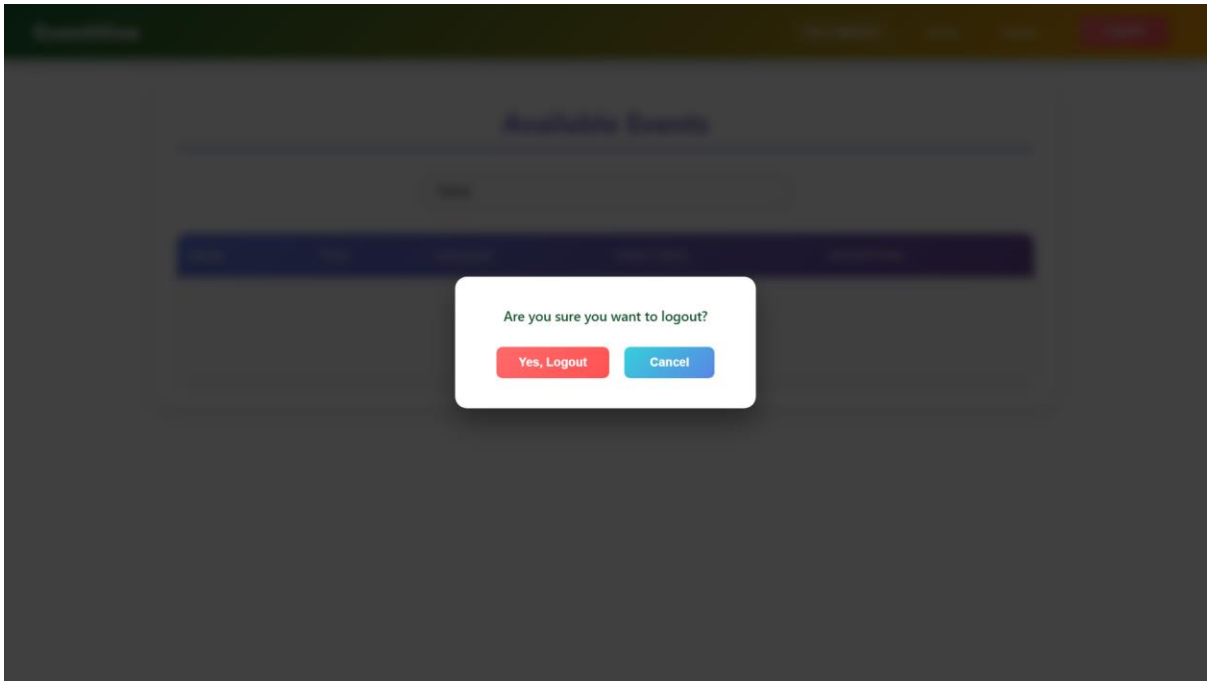
Additionally with search functionality based on event title.



If data is not available, then "Oops! No events found.” should be displayed.

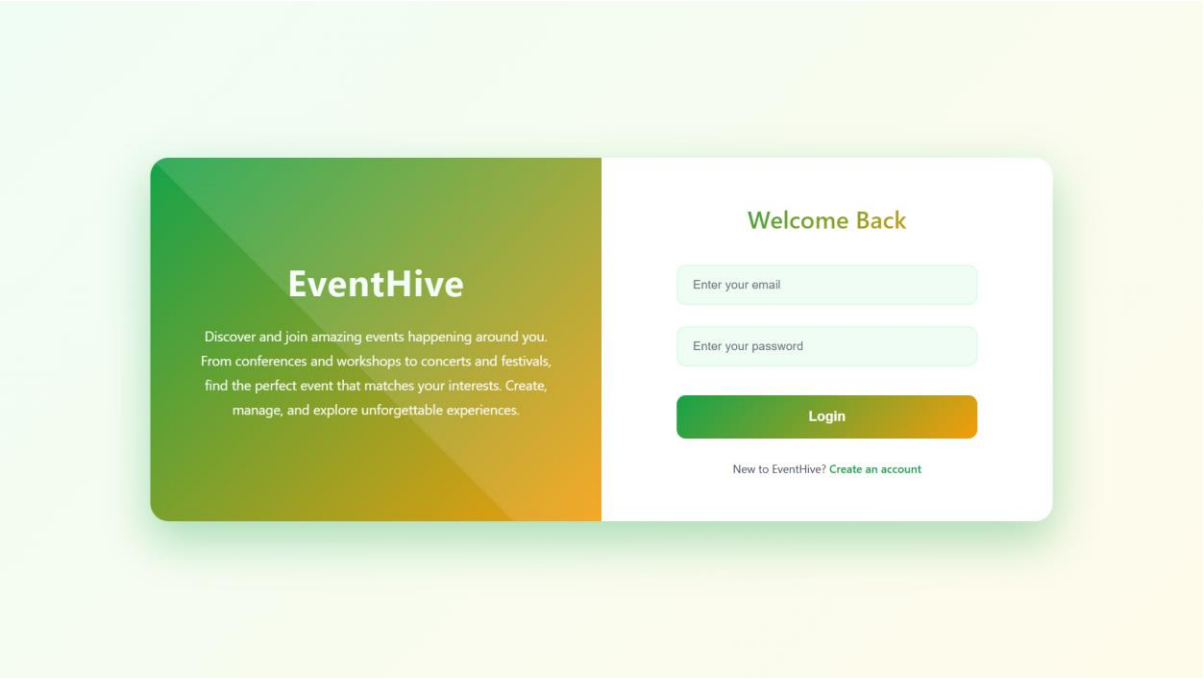


On clicking the "Logout" button, a pop-up should be displayed with confirmatory message to logout the user.



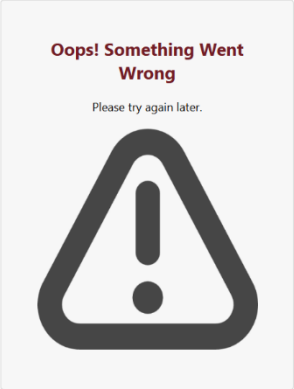
OnClicking “Cancel” will close the modal window displayed.

OnClicking "Yes, Logout" will navigate to the login component.



ErrorPage Component:

This page displays with “Oops! Something Went Wrong” message with the image (alert.png) provided in public folder.



Commands to Run the Project:
cd dotnetapp
Select the dotnet project folder
dotnet restore
This command will restore all the required packages to run the application.
dotnet run
To run the application in port 8080 (The settings preloaded click 8080 Port to View)
dotnet build
To build and check for errors
dotnet clean
If the same error persists clean the project and build again

To work with Entity Framework Core:
Install EF using the following commands:
dotnet new tool-manifest

dotnet tool install --local dotnet-ef --version 6.0.6

dotnet dotnet-ef --To check the EF installed or not

dotnet dotnet-ef migrations add "InitialSetup" --command to setup the initial creation of tables mentioned in DBContext

dotnet dotnet-ef database update --command to update the database

To Work with SQLServer:
(Open a New Terminal) type the below commands
sqlcmd -U sa
password: examlyMssql@123

1> create database appdb
2> go

>use appdb
>go

1> create table TableName(id int identity(1,1),.....)
2> go

Note:
The database name should be appdb.
Use the below sample connection string to connect the MsSql Server

private string connectionString = "User ID=sa; password=examlyMssql@123; server=localhost**;** Database=appdb; trusted_connection=false; Persist Security Info=False; Encrypt=False";

Platform Instructions to run the React project:
Step 1:
Use "cd reactapp" command to go inside the reactapp folder
Install Node Modules - "npm install**"**

Step 2:
Write the code inside src folder
Create the necessary components and enter the code

Step 3:
Command to run the project "npm start"

Step 4:
To run the test cases click the "Run Test Case" button.

<u>Note :</u>
Click PORT 8081 to view the result / output.
If any error persists while running the app, delete the node modules and reinstall them.

Cloud Deployment:

1. Introduction

1.1 Scope

The project involves deploying two separate applications (backend and frontend) to Azure App Services, integrating with Azure Key Vault for secrets management, using Azure SQL Database for data storage, and Azure Storage Account for storing uploaded images. The deployment will also include the creation and configuration of a service principal for secure access and necessary firewall rules for the SQL database.

Cloud Deployment

2.1 System Architecture

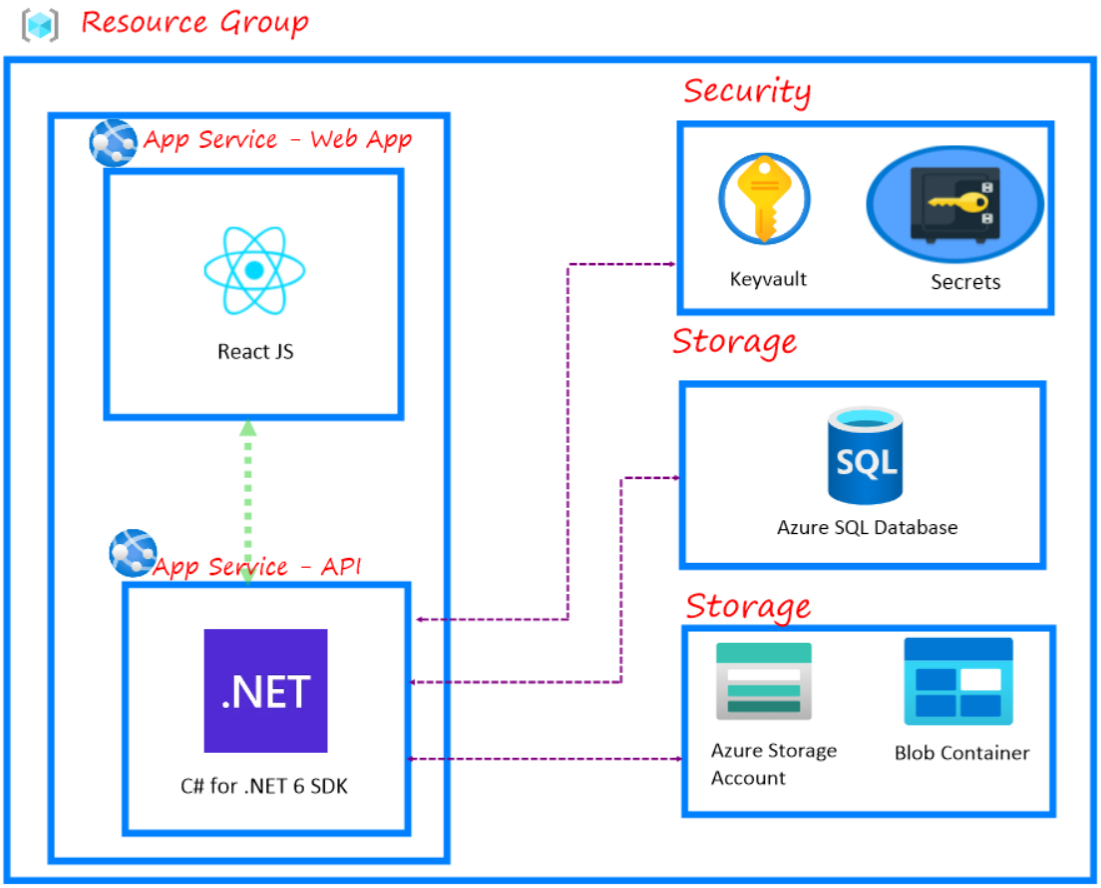
The system is divided into two main components: Backend and Frontend. Each component uses various Azure services to handle specific tasks.

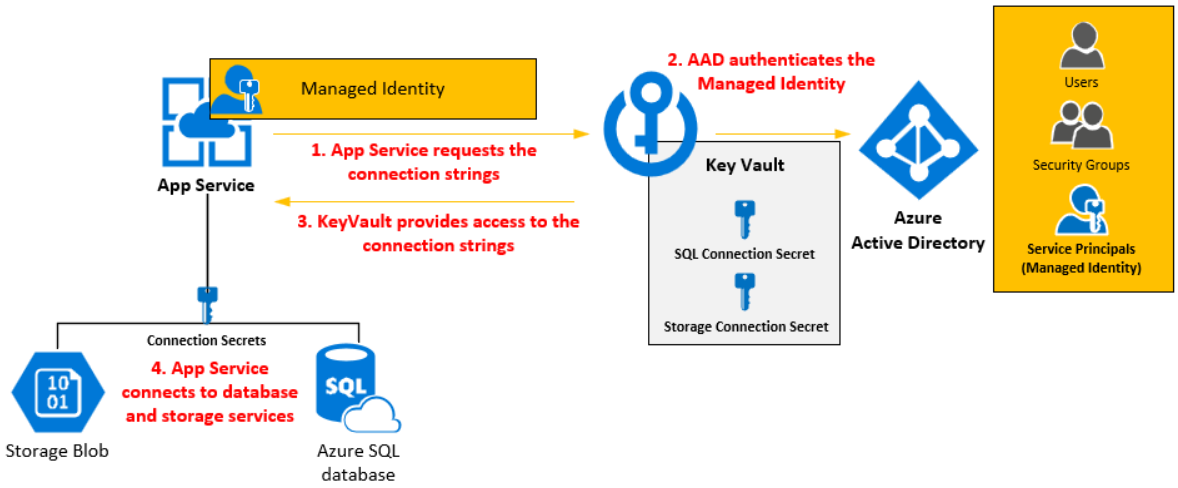
2.2 Backend Architecture

- **Azure App Service for .NET Backend**
  - App Name: "<Resource Group Name>bapp"
  - Configuration:
    - Deploy .NET backend using GitHub Actions
    - Configure to retrieve secrets from Azure Key Vault
    - Connect to Azure SQL Database using connection string from Key Vault
    - Store uploaded images in Azure Storage Account
- **Azure Key Vault**
  - Key Vault Name: <Resource Group Name>
  - Usage:
    - Store database connection strings, storage account keys, and other secrets
- **Azure SQL Database**
  - Database Name: "<Backend App Service Name>-database"
  - Configuration:
    - Apply firewall rules to allow required IP addresses
    - Store connection string in Azure Key Vault
- **Azure Storage Account**
  - Storage Account Name: <Resource Group Name>
  - Configuration:
    - Use blob storage to store uploaded images
    - Store storage account connection string in Azure Key Vault
- **Service Principal**
  - Usage:
    - Created for access management
    - Used in environment variables and GitHub secrets for deployments

- **2.3 Frontend Architecture**
- **Azure App Service for React Frontend**
  - App Name: “<Resource Group Name>fapp”
  - Configuration:
    - Deploy React frontend using GitHub Actions
    - Configure to retrieve secrets from Azure Key Vault

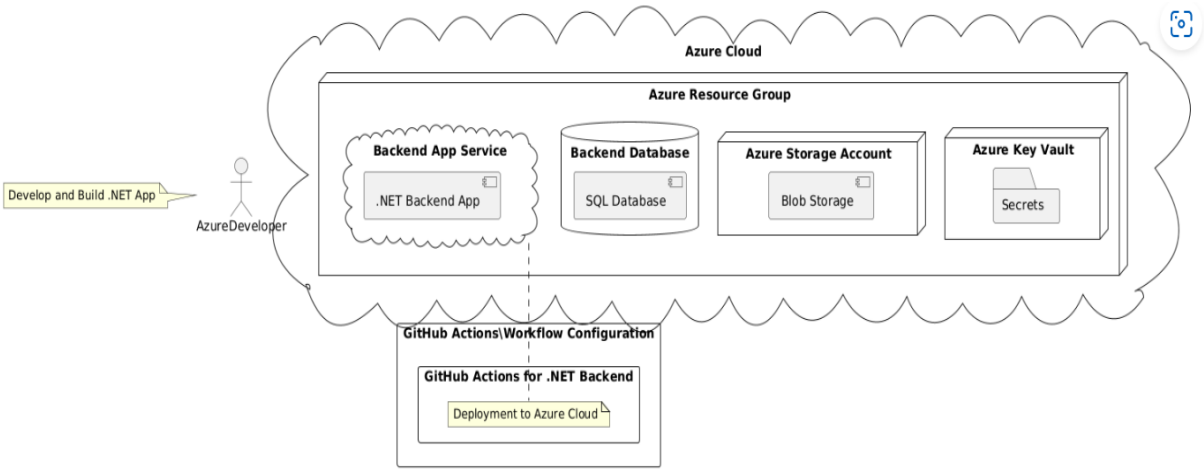
2.4 Architecture Diagram



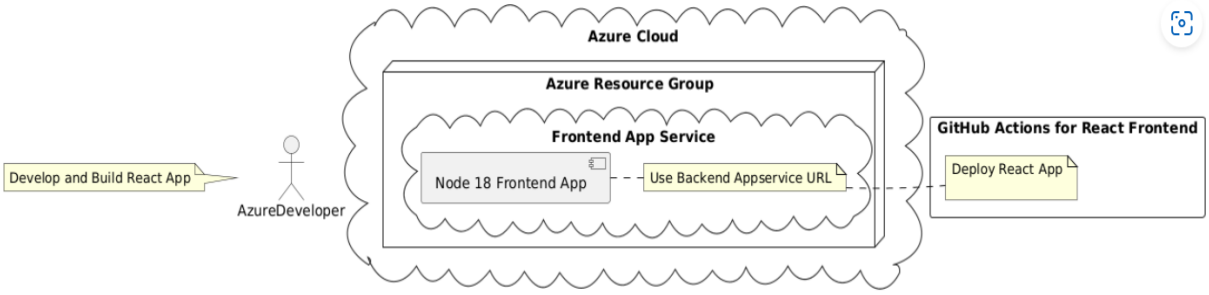


2.5 Flow Diagram

Backend Flow Diagram:



Frontend Flow Diagram:



3. Specific Requirements

3.1 Backend Services

Azure App Service for .NET Backend

- App Name: “<Resource Group Name>bapp”
- Deployment Steps:
  - Configure GitHub Actions workflow for deploying the .NET backend to bendapp24.
  - Run the GitHub Actions workflow.

Azure Key Vault

- Key Vault Name: <Resource Group Name>
- Configuration Steps:
  - Store necessary secrets (e.g., database connection strings, storage account keys) in edhubkvlt24.
  - Update backend application to fetch secrets from Key Vault.

Azure SQL Database

- Database Name: “<Backend App Service Name>-database”
- Configuration Steps:

    - Create an Azure SQL Database.
    - Add firewall rules to allow required IP addresses.
    - Store the connection string in Azure Key Vault.
    - Update the backend application to use the connection string from Key Vault.

Azure Storage Account

- Storage Account Name: <Resource Group Name>
- Configuration Steps:
    - Create an Azure Storage Account with blob storage.
    - Store the storage account connection string in Azure Key Vault.
    - Update the backend application to use the storage account for storing uploaded images.

Service Principal

- Configuration Steps:
    - Create a service principal and assign necessary permissions.
    - Store the service principal credentials as environment variables and GitHub secrets.
    - Update GitHub Actions workflows to use the service principal for deployments.

3.2 Frontend Services

Azure App Service for React Frontend

- App Name: "<Resource Group Name>fapp"
- Deployment Steps:
    - Configure GitHub Actions workflow for deploying the React frontend to "<Resource Group Name>fapp".
    - Run the GitHub Actions workflow.

3.3 GitHub Actions

GitHub Actions for .NET Backend

- Workflow Configuration:
    - Create a GitHub Actions workflow file to deploy the .NET backend to "<Resource Group Name>bapp"
    - Ensure the workflow includes steps for building the .NET application, logging into Azure, and deploying the application to Azure App Service.

GitHub Actions for React Frontend

- Workflow Configuration:
    - Create a GitHub Actions workflow file to deploy the React frontend to frontendapp24.

    - Ensure the workflow includes steps for building the React application, logging into Azure, and deploying the application to Azure App Service.

4.2 Integration with Azure Key Vault

- Steps:
    1. Store necessary secrets (e.g., database connection strings, storage account keys) in <Resource Group Name>
    2. Update both backend and frontend applications to fetch secrets from Key Vault.
    3. Deploy the applications.
- Expected Result: The applications should be able to retrieve and use secrets from the Key Vault successfully.

4.3 Azure SQL Database Configuration

- Steps:
    1. Create an Azure SQL Database.
    2. Add firewall rules to allow required IP addresses.
    3. Store the connection string in Azure Key Vault.
    4. Update the backend application to use the connection string from Key Vault.
    5. Deploy the backend application.
- Expected Result: The backend application should successfully connect to the SQL database using the connection string from Key Vault, and the SQL database should accept connections from the specified IP addresses.

4.4 Azure Storage Account Configuration

- Steps:
    1. Create an Azure Storage Account with blob storage.
    2. Store the storage account connection string in Azure Key Vault.
    3. Update the backend application to use the storage account for storing uploaded images.
    4. Deploy the backend application.
- Expected Result: The backend application should successfully store and retrieve images from the blob storage.

4.5 Service Principal Configuration

- Steps:
    1. Create a service principal and assign necessary permissions.
    2. Store the service principal credentials as environment variables and GitHub secrets.
    3. Update GitHub Actions workflows to use the service principal for deployments.
- Expected Result: The service principal should be successfully used for deployments and accessing resources, ensuring secure and seamless deployment.

General Instructions:

- You will be provided with an Azure Account Login Username and Password
- In the same account, a service principal's Client ID, Client Secret, and that Azure account's Subscription ID, Tenant ID will be provided.

Steps to be followed:

1. Create an Azure SQL database, a Storage Account, and a Keyvault

2. Complete the Backend Dotnet Development by mentioning the Azure SQL Database connection string and Azure Storage Account connection string along with the JWT Secret, JWT Valid issuer, JWT Valid Audience in the appsettings.json file. These connection strings and JWT Credentials totally 5 secrets must be stored in Azure Keyvault as secrets as mentioned below and it shouldn't be disclosed or called upon anywhere in the application development.
    - ASPNETCORE_ENVIRONMENT : Development
    - KeyVaultConfiguration__BlobSecret : blobsecret
    - KeyVaultConfiguration__ConnectionStringSecretName: sqlconnectionstring
    - KeyVaultConfiguration__JWTSecret : JWTSecret
    - KeyVaultConfiguration__JWTValidAudience : JWTValidAudience
    - KeyVaultConfiguration__URL : https://<Your Keyvault Name>.vault.azure.net

3. Once completed the Dotnet Backend Development, complete the Frontend React Development

4. Do the platform testing in 8080 and 8081 for backend and frontend respectively

5. Deploy two Azure App services for backend and frontend

6. In the Backend Dotnet application Azure App service, mention the Keyvault URL, all the 5 Keyvault secrets, also additional environmental variable 'ASPNETCORE' with the value 'Development' totally 7 are to be given as Azure App settings Environmental Variables.

7. Complete the Backend Deployment to Azure App service via GitHub Actions which can use the Backend Publish Profile as Azure Web App Publish Profile can be used. So this should be stored in GitHub Secrets.

8. After the successful deployment of Dotnet backend application to respective Azure App service, build the Front end React application with the Azure Backend App service URL

9. Once completed the build process successfully, push the Frontend code with 'build' folder to GitHub.

10. Deploy the Frontend application with the already built 'build' folder to the second Azure App service which will be in reactapp build folder.

11. In the deployed Front end React Azure App service, mention in the App settings Environmental variable wih the following Backend App Service URL:

    - API_BASE_URL: "https://<Your Backend App Service Name>.azurewebsites.net"

12. For the React application site access, give the following command in the Frontend App Service Configuration App Settings Start up command :

    pm2 serve /home/site/wwwroot --no-daemon --spa